Hima — QA Release Testing SOP & Roadmap

Standard operating procedure for testers: what to test **before** release and **after** release

Innovfix Pvt. Ltd. · Hima App · v1 · 2026-07-14 · Owner: Yuvanesh (Tech Lead)

> **How to use this doc.** For every release: run **Phase 1–4 (before release)** and only ship when the sign-off gate passes; then run **Phase 5 (after release)** on production. BEFORE = pre-release on staging/APK. AFTER = post-release on live prod. MUST-PASS = a failure blocks release.

1. Testing Phases (the process)

Phase	When	What
0 · Build sanity	New APK arrives	Installs, opens, no crash on launch, correct version number, login screen loads.
1 · Regression	Before release	Every module's core flows still work (Section 3 checklists).
2 · New-feature	Before release	Deep-test whatever changed in THIS release (from the branch/changelog).
3 · Critical-path	Before release	Money + calls deep test (Section 4) — zero tolerance.
4 · Sign-off gate	Before release	No Sev-1/Sev-2 open. Section 6 checklist signed.
5 · Post-release smoke + monitor	Right after release	Prod smoke on 2–3 real devices + watch live metrics for 2–4h (Section 5).
6 · Rollback watch	First 24h	If a rollback trigger hits (Section 5), revert/hotfix immediately.

Environments & test accounts (set up once)

- **Devices:** ≥2 Android phones (1 low-end, 1 high-end), on WiFi **and** mobile data. Test the **Play Store / Internal Testing** build, not only sideload (sideload can pass while Play build fails — always confirm on the real distribution channel).
- **Accounts:** a **male caller** (with test coins), a **female creator** (verified, with bank/UPI + PAN), and a **fresh new user** (for onboarding). Keep a second creator for block/DND pairing tests.
- **Money:** use small real amounts (₹1–₹49) for real gateway tests — never assume; a coin credit must be verified end-to-end.

2. Bug severity & the release gate

Severity	Examples	Blocks release?
Sev-1 Critical	Coins credited without payment / free coins · double billing · double payout · app crash on launch · calls don't connect · can't log in · money lost	🔴 **YES — hard block**
Sev-2 Major	A gateway fails · withdrawal stuck · matcher connect-rate drops · KYC/verify broken · chat not delivering	🔴 **YES**
Sev-3 Minor	UI glitch, wrong text, non-blocking edge case	🟡 **Ship with a follow-up ticket**

3. BEFORE RELEASE — module-by-module roadmap BEFORE

3.1 Login · Auth · Onboarding

✓	Test
☐	New user: enter mobile → OTP received → verify → profile setup → lands on home.
☐	Wrong OTP rejected; resend OTP works; OTP expiry handled.
☐	Existing user login + logout + re-login (token refresh) works.
☐	Profile update: name (one-time name-change rule), avatar, interests (optional), language.

✓	Test
☐	Gender selection correct; male vs female home screens differ correctly.
☐	AI onboarding chat triggers for new male users; icebreaker shows.
☐	Force-update: a below-min-version user is prompted/blocked per `individual_app_update`.
☐	Delete account flow works; deleted user can't log back in.
☐	**NEW** Keyboard visibility: after entering the mobile number, the **mobile number field is fully visible** above the keyboard (not hidden); and on the OTP step the **6-digit OTP boxes are fully visible** above the keyboard. Both steps, low-end + high-end device. *(QA #4)*

3.2 Creator verification / KYC

✓	Test
☐	Voice verification: record voice → female auto-approve / male-voice reject.
☐	PAN KYC (Cashfree Secure-ID, PaySprint fallback): valid PAN passes, invalid rejected.
☐	Bank + UPI details save + validate; can't withdraw before verification.

3.3 Calls (audio + video) MUST-PASS parts

✓	Test
☐	**Random match:** caller taps random → matched to an online creator in his language → rings.
☐	**Connect:** creator answers → Agora audio/video connects both ways → timer starts (`started_time`).
☐	**Billing:** coins deduct per minute from caller; creator income accrues correctly; amounts match the rate.
☐	**No double-billing** (rapid reconnects / weak network don't charge twice).
☐	**Coins exhaust:** server force-ends the call when caller's coins hit 0.
☐	**Direct call** to a specific creator; recent/favourite call works.
☐	**Reject:** creator rejects → caller sees "busy/unavailable"; 3-reject cool-down applies.
☐	**Missed call:** unanswered → missed_call count + push to creator.
☐	**Video & audio both**; switch between; low-network behaviour (call doesn't hang forever).
☐	Call ends cleanly on either side; end reason recorded; earnings shown to creator.
☐	**NEW** **Network drop while connecting:** caller's network dies mid-ring → the **creator's ring auto-ends** within ~45s (no connected-but-silent call that has to be hung up manually). *(QA #2)*
☐	**NEW** **Ring stops on cancel — app closed:** creator's app is closed/backgrounded and rings → caller cancels → the ringing/vibration **stops immediately** (no ghost ring to dismiss by hand). *(QA #6)*
☐	**NEW** **Call history label:** a connected + completed call always shows its **duration** (even a very short call) in Call History — it is **never labelled "Missed"**. Genuinely unanswered calls still show "Missed". *(QA #10)*
☐	**NEW** **No duplicate/ghost calls on reopen:** after the creator closes and reopens the app, previously received incoming/missed calls do **not** re-appear or re-ring as new incoming calls (queued pushes for already-ended calls are dropped). *(QA #7)*

3.4 Do-Not-Disturb & Blocking

✓	Test
☐	Creator turns DND on → she gets NO incoming calls/chat pings → shows unavailable → stays on until she turns it off.
☐	Call-block: blocked caller can't call her at all; matcher never pairs them again.
☐	Chat-block: blocked caller can't message; existing chat stops.
☐	Both honoured silently (no "you are blocked" popup) across call + chat + matcher.
☐	**NEW** After blocking a contact, that conversation **moves to the bottom** of the chat list (and shows the Blocked badge). *(QA #8)*

3.5 Coins & Payments MUST-PASS

Zero tolerance: coins must be credited ONLY after a verified real payment, exactly once. Test EACH live gateway with a small real amount.

✓	Test
☐	**Cashfree:** buy a pack → pay → coins credited exactly once; check `transactions.add_coins`.
☐	**PhonePe:** same end-to-end.
☐	**Razorpay / UPI link:** same end-to-end.
☐	**Payment FAIL / cancel:** NO coins credited; balance unchanged; clean error to user.
☐	**Webhook replay / forged:** a repeated or unverified webhook does NOT re-credit or free-credit coins.
☐	Offers, trial coins, coupons apply the correct discounted price + coins.
☐	Gateway switch/maintenance flag: when a gateway is off, the app falls back correctly.

3.6 Autopay / Subscription

✓	Test
☐	₹1 trial sheet shows to never-subscribed; mandate created (UPI, GPay-compatible).
☐	Daily coin claim works; recurring ₹299 charge fires; subscription status correct.
☐	Cancel subscription works; CHARGE-fail vs AUTH-fail push messages correct.

3.7 Withdrawals / Payouts MUST-PASS

Zero tolerance: a creator must be paid the correct amount exactly once — never double-paid, never a phantom deduction.

✓	Test
☐	Request withdrawal → balance deducts → Cashfree payout (IMPS) → status Pending→Paid → money reaches bank.
☐	**No double-pay** (rapid taps / retries don't pay twice).
☐	Failed/rejected transfer → balance rolled back correctly.
☐	PAN gate + TDS applied where required; withdrawal blocked without PAN.
☐	Min-amount, bank-detail-missing, and reversal (auto-refund coins) edge cases.

3.8 Creator earnings, tiers & bonuses

✓	Test
☐	Income from calls shows correctly on creator dashboard; balance + `total_income` match.
☐	Tier/score updates; call-bonus tiers pay the right bonus; icebreaker prompts show.
☐	Creator warnings issue + auto-disable + auto-unblock work; star-creator rates apply.
☐	NEW **Call-duration match:** the creator's **Call History duration = Earnings/Transactions duration = the real connected talk time** (measured answer→hangup, with no ring/setup time included). *(QA #12)*
☐	NEW **Every connected call is credited:** a call that connected (`started_time` set) credits the creator for its real talk time **even if the caller's app didn't cleanly report the end** (force-close / network drop) — no answered call is left with ₹0 income. *(QA #11)*

3.9 Chat & Social (real-time)

✓	Test
☐	Send/receive message in real time (both directions); single→blue ticks (read receipts).
☐	Reactions add/update; message delete (archived server-side) + reflects for both.

✓	Test
☐	Bad-word message filtered/handled; presence/typing indicators show.
☐	Attachments upload; gifts send + deduct coins + credit creator; friend requests flow.
☐	Open-chat-from-notification deep-links to the right thread.
☐	NEW Image and audio **attachment options are removed** from the chat composer (emoji available in their place). *(QA #3)*

3.10 Notifications

✓	Test
☐	Push received (FCM + OneSignal): incoming call, new message, friend request, creator-online.
☐	Tapping a notification opens the correct screen; DND suppresses pings.
☐	Notification click/conversion tracking records (if part of this release).
☐	NEW Tapping a **video**-call notification starts a **video** call (not audio) when both call types are enabled. *(QA #9)*
☐	NEW Rejected calls are **not** shown/pushed as "missed"; missed-call notification text shows the correct call status (not both audio & video). *(QA #5)*

3.11 Growth, Games & misc

✓	Test
☐	Install referrer / tracking link / deep link attribute correctly (if changed).
☐	IPL rooms create/join/leave/react; Ludo invite + play (if in release).
☐	Reports/ratings/reviews; app-store rating prompt triggers per rules.

3.12 Admin panel (backend) BEFORE

✓	Test
☐	NEW Block a user/creator with a reason → reopen the account → the **block reason and blocked status/icon are still shown** (retained for audit). *(QA #1)*

4. CRITICAL MUST-PASS scenarios (money + calls) MUST-PASS

These four must be re-verified **every release**, end-to-end, on the real Play build. A single failure = **do not ship**.

1. **Pay → get coins, once.** Real payment on each live gateway → exact coins credited once. Fail/forged → no credit.
2. **Call → correct billing.** Connect → per-minute deduct + creator income correct → no double-bill → force-end on 0 coins.
3. **Withdraw → paid once.** Payout reaches bank, correct amount, no double-pay, rollback on failure.
4. **Matcher + safety.** Random call connects (~normal rate), and DND/blocked pairs are never matched.

5. AFTER RELEASE — production verification & monitoring AFTER

5.1 Smoke test on prod (first 30 min, 2–3 real devices)

✓	Test on the LIVE app
☐	Fresh install from Play → open → login/OTP works.
☐	One real random call connects both ways.
☐	One real small coin purchase credits coins.
☐	One real chat message delivers + blue ticks.

✓	Test on the LIVE app
☐	A test withdrawal request is accepted (status pending).

5.2 Watch these live metrics for 2–4 hours

Metric	Healthy	Rollback trigger
Connect rate (started_time)	~45–52%	Sudden drop >10 pts vs same slot yesterday
Revenue (add_coins/hr)	≈ prior day same hour	Sharp fall vs baseline (not a demand dip)
Gateway success %	Cashfree ~56%, PhonePe ~51%	Any gateway drops >10 pts
Payout %	~40%	Spikes (income leak / ghost calls)
Crashlytics crash-free	>99%	New crash cluster after release
Error log (laravel.log)	baseline noise	New error spiking (e.g. class-not-found, 500s)

> **Version-aware check:** compare the NEW version's users (by `current_version`) vs the stable version on connect rate, ARPU, and gateway success — a bad release shows as the new version underperforming (like v1109 did). Roll out in staged % and watch before 100%.

5.3 Rollback / hotfix criteria

- Any **Sev-1** in production (money loss, free coins, double-pay, mass crash, calls not connecting) → **halt rollout + hotfix/rollback immediately.**
- Connect rate or revenue drops sharply and it's traced to the release (not external) → roll back the staged %.
- A gateway or payout breaks → flip its kill-switch (`gateway_config`) and fix.

6. Sign-off checklist (gate before release)

✓	Item
☐	Build sanity passed (installs, opens, correct version, no launch crash) — on the **Play/Internal** build.
☐	All 4 **MUST-PASS** critical scenarios passed end-to-end.
☐	Regression (Section 3) done for all modules; new-feature deep-tested.
☐	**Zero open Sev-1 / Sev-2.** All logged in the bug tracker with severity.
☐	Tested on ≥2 devices, WiFi + mobile data.
☐	Post-release smoke + monitoring plan assigned (who watches, for how long).
☐	Rollback owner + kill-switches identified. Signed by QA lead + Tech lead.

5. AFTER RELEASE — production verification & monitoring AFTER

5.1 Smoke test on prod (first 30 min, 2–3 real devices)

✓	Test on the LIVE app
☐	Fresh install from Play → open → login/OTP works.
☐	One real random call connects both ways.
☐	One real small coin purchase credits coins.
☐	One real chat message delivers + blue ticks.
☐	A test withdrawal request is accepted (status pending).

5.2 Watch these live metrics for 2–4 hours

Metric	Healthy	Rollback trigger
Connect rate (started_time)	~45–52%	Sudden drop >10 pts vs same slot yesterday
Revenue (add_coins/hr)	≈ prior day same hour	Sharp fall vs baseline (not a demand dip)
Gateway success %	Cashfree ~56%, PhonePe ~51%	Any gateway drops >10 pts
Payout %	~40%	Spikes (income leak / ghost calls)
Crashlytics crash-free	>99%	New crash cluster after release
Error log (laravel.log)	baseline noise	New error spiking (e.g. class-not-found, 500s)

Version-aware check: compare the NEW version's users (by `current_version`) vs the stable version on connect rate, ARPU, and gateway success — a bad release shows as the new version underperforming (like v1109 did). Roll out in **staged %** and watch before 100%.

5.3 Rollback / hotfix criteria

- Any **Sev-1** in production (money loss, free coins, double-pay, mass crash, calls not connecting) → **halt rollout + hotfix/rollback immediately.**
- Connect rate or revenue drops sharply and it's traced to the release (not external) → roll back the staged %.
- A gateway or payout breaks → flip its kill-switch (`gateway_config`) and fix.

6. Sign-off checklist (gate before release)

✓	Item
☐	Build sanity passed (installs, opens, correct version, no launch crash) — on the **Play/Internal** build.
☐	All 4 **MUST-PASS** critical scenarios passed end-to-end.
☐	Regression (Section 3) done for all modules; new-feature deep-tested.
☐	**Zero open Sev-1 / Sev-2.** All logged in the bug tracker with severity.
☐	Tested on ≥2 devices, WiFi + mobile data.
☐	Post-release smoke + monitoring plan assigned (who watches, for how long).
☐	Rollback owner + kill-switches identified. Signed by QA lead + Tech lead.

RELEASE TEST APPENDIX ACX

Local Call-Audio Extraction Foundation

SOP revision 1.1 - 2026-07-17 - Applies to development APK before any transcription provider is connected.

Release gate

ACX-001, 002, 003, 004, 005, 007 and 008 are MUST-PASS on real devices. A compile/unit-test pass alone does not authorize production rollout. Attach logs, profiler screenshots and defect/retest evidence.

ACX-001 - Male local microphone extraction	MUST-PASS / P0
Preconditions / build / role	Development APK; two real devices; male caller and female creator; call connects normally.
Test data	Speak a fixed 65-second script on the male device; female remains quiet.
Steps	1. Start an audio call. 2. Speak before and across 30 s and 60 s boundaries. 3. End normally. 4. Capture filtered AudioExtractor logs.
Expected result	Call audio remains clear. Male device reports monotonic chunks with 30 s max and 1 s overlap after the first. Female local track does not contain male playback. No file/network/AI/warning appears.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

ACX-002 - Female local microphone extraction and attribution	MUST-PASS / P0
Preconditions / build / role	Same two-device build; connected audio call.
Test data	Female speaks fixed script while male stays quiet; then both alternate turns.
Steps	1. Record female-only speech for 35 s. 2. Alternate speakers. 3. Compare each device's local metadata and audible call.
Expected result	Female device marks local speech. Male device marks only its own spoken turns, not remote playback. No speaker-attribution crossover or call-audio regression.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

ACX-003 - Manual mute privacy boundary	MUST-PASS / P0
Preconditions / build / role	Connected call; metadata logs available.
Test data	Speak A while live, mute and speak PRIVATE, unmute and speak B.
Steps	1. Speak A. 2. Tap mute. 3. Speak for at least 10 s while muted. 4. Unmute and speak B. 5. End call.
Expected result	Peer hears A and B only. Pre-mute partial closes; no extracted duration advances during mute; post-unmute starts a clean non-overlapping boundary. PRIVATE is never present in any future payload.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

ACX-004 - Phone/audio-focus interruption	MUST-PASS / P0
Preconditions / build / role	Connected call; second incoming cellular or VoIP/audio-focus event available.
Test data	Speak before, during and after the interruption.
Steps	1. Trigger interruption. 2. Speak while HIMA is held. 3. End interruption. 4. Resume HIMA speech. 5. Repeat once.
Expected result	Extraction pauses with outgoing mic, resumes with a clean boundary, sequence remains monotonic, call does not crash, duplicate workers are not created and held/private speech is excluded.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

Long-Call, VAD and Failure Coverage

Environment: development APK, two real devices, Wi-Fi and mobile data, wired/Bluetooth where available.

ACX-005 - Boundary and hour-long stability	MUST-PASS / P0
Preconditions / build / role	Two charged test accounts; devices on power; Android Studio profiler or equivalent.
Test data	At least 65 min call; scripted words spanning each sampled 30 s boundary.
Steps	1. Run call for 65 min. 2. Sample logs at start, 30 min and 60 min. 3. Exercise speaker, wired and Bluetooth routes. 4. End normally.
Expected result	No 30 s recording stop, missing tail, duplicate-only final chunk, ANR, OOM or increasing worker count. Windows stay <=960 KB; sequence/timestamps are monotonic; audible call quality remains normal.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

ACX-006 - VAD shadow labels across languages	P1
Preconditions / build / role	Connected call in a quiet room, then normal background noise.
Test data	English plus all 11 HIMA languages and code-switching; silence; soft speech; loud speech; fan noise.
Steps	1. Speak each sample naturally. 2. Include quiet and noisy periods. 3. Compare hasSpeech/voiced ratio to the known speaking periods.
Expected result	VAD is language-independent and labels ordinary speech without modifying/cutting PCM. Silence is normally false. Noise false positives are documented; any speech false negative blocks using VAD to skip future API calls.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

ACX-007 - Teardown, reconnect and permission failure	MUST-PASS / P0
Preconditions / build / role	Connected call; ability to toggle network and revoke RECORD_AUDIO.
Test data	Wi-Fi/mobile switch, temporary loss, peer leave, local hangup, permission revoke.
Steps	1. Reconnect network twice. 2. End via peer and local paths. 3. Repeat with permission revoke. 4. Start a fresh call after each case.
Expected result	Observer is not duplicated on reconnect; dispose is safe from leaveChannel/onDestroy; no extractor thread survives teardown; next call captures normally; existing call failure handling remains authoritative.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

ACX-008 - Privacy, no-spend and regression gate	MUST-PASS / P0
Preconditions / build / role	Fresh development APK; proxy/file inspection tools; admin access for read-only verification.
Test data	One 2-minute call containing harmless test speech.
Steps	1. Inspect app private cache/database before/during/after. 2. Inspect network requests. 3. Check admin/warnings. 4. Verify call settlement/rating flow.
Expected result	Zero audio files, uploads, transcript rows, provider calls, AI cost, admin audio, moderation results or warnings. Existing connection, mute, timer, hangup, settlement and rating flows still complete.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

Creator-online Notification Call-Type Routing

SOP revision 1.2 - 2026-07-17 - Production payload hotfix. Applies to both instant and scheduled OneSignal creator-online senders.

Release gate	CTN-001, CTN-002 and CTN-003 are MUST-PASS. Do not send a production test notification without owner approval. Record payload evidence, device evidence, defect ID and retest status.

CTN-001 - Instant video notification routes to video

MUST-PASS / P0

Preconditions / build / role	Backend hotfix on all three nodes; instant flag ON; current production APK; male caller and verified female creator; creator has both audio and video enabled.
Test data	Use an approved test creator/caller pair. Enable video last so the instant alert advertises video.
Numbered steps	1. Capture the generated OneSignal data object without exposing tokens. 2. Confirm call_type=video. 3. Tap the alert on the male device. 4. Observe the connecting and in-call screen.
Expected result	Payload contains source=creator_online, the intended creator ID and call_type=video. Tap opens the video connecting flow and joins a video call; it never starts audio.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

CTN-002 - Scheduled fallback preserves video type

MUST-PASS / P0

Preconditions / build / role	QA/demo environment or an explicitly approved controlled production test; instant path disabled or deliberately made to leave the queue; scheduled creator processor active; both creator call types enabled.
Test data	Queue one creator-online video event for the approved pair.
Numbered steps	1. Let creators:process-notifications claim the queue row. 2. Capture redacted payload evidence. 3. Confirm call_type=video. 4. Tap the resulting alert.
Expected result	Scheduled payload and human text agree on video. Tap opens video. The fallback no longer loses call_type or silently selects audio.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

CTN-003 - Audio notification remains audio

MUST-PASS / P0

Preconditions / build / role	Current APK; creator has both types enabled; approved instant and scheduled test paths available.
Test data	Enable audio last so the advertised type is audio.
Numbered steps	1. Exercise instant sender and scheduled sender separately. 2. Verify call_type=audio in each redacted payload. 3. Tap each notification.
Expected result	Both alerts open the audio connecting flow. Video routing fix does not invert or break existing audio behavior.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

CTN-004 - Availability and stale-type fallback	P1
Preconditions / build / role	Current APK; controlled accounts; ability to change creator audio/video availability before tap.
Test data	Video-only, audio-only, both-off, and a payload whose advertised type becomes unavailable before tap.
Numbered steps	1. Test video-only and audio-only alerts. 2. Turn the advertised type off before tapping while leaving the other type on. 3. Test both-off. 4. Record resulting screen/message.
Expected result	Available advertised type is honored. If it became unavailable, app uses the remaining available type. Both-off does not launch a billable call and shows the existing unavailable handling.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

CTN-005 - Legacy payload compatibility	P1
Preconditions / build / role	Debug/QA injection only; current APK; do not send a real provider notification.
Test data	Creator-online payload omitting call_type, representing an old queued/provider payload.
Numbered steps	1. Inject/open the legacy payload in QA. 2. Test audio-only, video-only and both-enabled creator states. 3. Confirm app remains stable and clears stored notification routing keys after use.
Expected result	No crash. Audio-only and video-only still open their available type; both-enabled retains the documented backward-compatible audio fallback only for legacy payloads.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

CTN-006 - Production smoke and regression evidence	P1 / AFTER
Preconditions / build / role	Approved production smoke window; all three live hashes match; PHP-FPM 8.2/8.4 active; owner approval for any outward test notification.
Test data	One approved audio alert and one approved video alert; no customer accounts or PII in evidence.
Numbered steps	1. Verify live source/hash and service health. 2. Send only the approved test alerts. 3. Tap each on a current APK. 4. Monitor Laravel/PHP errors and creator-online send failure rate for 30 minutes. 5. Record rollback decision.
Expected result	Audio opens audio and video opens video. No new PHP errors, send failures, duplicate sends, latency regression, database work or availability regression. Reverse the two payload lines if a rollback trigger occurs.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

RELEASE TEST APPENDIX RNG

Incoming Ring Stops After Caller Disconnect

SOP revision 1.3 - 2026-07-17 - Authenticated Redis heartbeat plus legacy fallback. No user_calls migration and no call-row write from liveness polling.

Release gate	RNG-001 through RNG-005 are MUST-PASS before rollout. Run network, Redis-failure and load cases in QA. Production calls or notifications require separate owner approval. Record build, timing, redacted evidence, defect and retest status.

RNG-001 - Original ghost-ring reproduction on legacy APK

MUST-PASS / P0

Preconditions / build / role	QA backend patch; production-equivalent APK that does not send call_ring_heartbeat; caller and recipient test accounts; incoming screen polling active.
Test data	Audio and video, both call directions, caller network disabled during pre-answer setup.
Numbered steps	1. Start the call. 2. Confirm recipient is ringing. 3. Disable caller network before Answer. 4. Leave recipient untouched. 5. Measure from call creation and from network loss. 6. Repeat with caller app killed.
Expected result	Recipient ringtone, Telecom call and incoming notification all stop by 47.5 seconds from creation. No Answer screen remains and no old ring reappears. No coins are deducted because started_time remains empty.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

RNG-002 - Authenticated heartbeat stops a disconnected caller quickly

MUST-PASS / P0

Preconditions / build / role	QA backend patch; new APK containing authenticated heartbeat; valid JWT; clocks synchronized for evidence only.
Test data	Drop Wi-Fi, mobile data, airplane mode and process kill separately at 5-10 seconds of ringing.
Numbered steps	1. Start ringing and capture successful heartbeat responses. 2. Trigger one disconnect mode. 3. Measure last accepted heartbeat to recipient teardown. 4. Repeat each mode for audio/video and both caller genders.
Expected result	Recipient teardown occurs in 15-18 seconds after the last accepted heartbeat. check_call_alive returns alive=false, ringable=false and reason=not_answered. The call row is not changed by the liveness poll.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

RNG-003 - Answer and heartbeat timeout boundary

MUST-PASS / P0

Preconditions / build / role	QA backend patch and new APK; controllable latency or packet loss; both devices online initially.
Test data	Answer attempts immediately before and after the 15-second stale boundary, including a delayed heartbeat response.
Numbered steps	1. Delay caller heartbeat traffic. 2. Tap Answer at 14, 15 and 16 seconds in separate calls. 3. Observe accept-in-flight behavior, Agora join, server response and billing state. 4. Repeat 20 times.
Expected result	An Answer already in flight is not dismissed by a late poll callback. A call declared unreachable does not open a billable empty room. No row contains contradictory started and terminal state; no double ring occurs.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

RNG-004 - Heartbeat authorization and ownership	MUST-PASS / P0
Preconditions / build / role	API test client in QA; one male-initiated and one female-initiated call; caller, recipient and unrelated JWTs.
Test data	Missing token, invalid token, recipient token, unrelated token, caller token, invalid and nonexistent call IDs.
Numbered steps	1. POST each credential/call combination. 2. Verify status and response. 3. Inspect Redis keys without exposing tokens. 4. Confirm no user_calls write occurred.
Expected result	Only the actual initiator can refresh the heartbeat. Missing/invalid authentication is rejected, recipient/unrelated users receive forbidden, malformed IDs are rejected, and no database row is mutated.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

RNG-005 - Delayed push, recovery and no resurrection	MUST-PASS / P0
Preconditions / build / role	QA backend patch; new APK; ability to delay FCM delivery and background/foreground the recipient app.
Test data	Fresh ring, heartbeat-stale ring, 30-second age-guard ring and 45-second legacy-terminal ring.
Numbered steps	1. Delay each incoming push. 2. Trigger foreground pending-call recovery. 3. Deliver the old push after each boundary. 4. Reopen the app. 5. Attempt a new unrelated call.
Expected result	Only a fresh unanswered call is surfaced. ringable=false blocks stale/started calls, pending recovery does not resurrect them, and a later new call rings normally.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

RNG-006 - Redis failure and backward-compatible fallback	P1
Preconditions / build / role	QA only; controlled Redis unavailability or injected Cache exception; legacy and new APKs.
Test data	Cache read failure, write failure, recovery during ring and heartbeat endpoint timeout.
Numbered steps	1. Inject each failure. 2. Start a call and attempt heartbeats. 3. Observe API status, logs and recipient ring. 4. Restore Redis and repeat.
Expected result	Cache failure does not end a legitimate call early and does not return a server 500. The new client degrades to the 45-second legacy terminal response; recovery needs no database repair.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

RNG-007 - Capacity and database regression gate	P1 / LOAD
Preconditions / build / role	Staging or isolated production-equivalent environment; representative concurrency; query and Redis telemetry.
Test data	At least 100 concurrent ringing sessions for five minutes plus current check_call_alive traffic.
Numbered steps	1. Generate 2.5-second authenticated heartbeats and both-side liveness polls. 2. Record p50/p95/p99 latency, Redis ops, PHP CPU/RSS and DB queries. 3. Verify query plan. 4. Compare baseline.
Expected result	Heartbeat is bounded to about 24 requests/minute per ringing caller, uses one caller-auth lookup plus one user_calls PK lookup and one Redis SET, creates no migration or call-row write, and does not materially regress API/DB capacity.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

RNG-008 - Three-node production smoke and rollback	P1 / AFTER
Preconditions / build / role	Separate owner approval for node patch, PHP-FPM reload and any real-device call; all three node hashes verified first.
Test data	One approved legacy-client call and one approved new-client call; no customer data in evidence.
Numbered steps	1. Verify route/service/controller hashes on app-1/2/3. 2. Verify PHP-FPM 8.2/8.4. 3. Run approved calls. 4. Monitor 4xx/5xx, Redis latency and PHP errors for 30 minutes. 5. Record rollback decision.
Expected result	All nodes behave identically, ordinary calls connect normally, legacy teardown meets 47.5 seconds, new teardown meets 15-18 seconds, and no billing, availability, database-write or service-health regression appears.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

RELEASE TEST APPENDIX CMR-UI

Call Moderation Compact Controls

Demo admin UI revision - 2026-07-17 - Blade-only presentation change; control routes and behavior remain unchanged.

CMR-UI-001 - Desktop one-line control rail	MUST-PASS / P0
Preconditions / build / role	Demo admin account with Call Moderation access; desktop viewport at least 1200px; moderation settings visible.
Test data	Master On and Off states, auto-warning On and Off states, minimum version 0 and a positive version code.
Numbered steps	1. Open Call Moderation Review. 2. Inspect the first control area at 1200px and 1920px widths. 3. Verify labels, values and state colors. 4. Expand Recent changes.
Expected result	Moderation, Auto warning and Minimum app version appear in one compact row. Each control has a short label, current value and Save button. No former large headings or explanatory paragraphs remain. Recent changes stays available below the row without clipping or overlap.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

CMR-UI-002 - Control behavior and authorization regression	MUST-PASS / P0
Preconditions / build / role	Authorized demo admin; separate unauthorized or insufficient-role session; known initial control values; no real call required.
Test data	Toggle each switch once and restore it; version values 0, valid positive integer, negative value and value above 2147483647.
Numbered steps	1. Change each control and press its Save button. 2. Confirm the warning prompt. 3. Reload and verify persistence/history. 4. Test invalid version boundaries. 5. Repeat update requests without authorization.
Expected result	Existing PATCH routes, CSRF, confirmations, permission checks, validation and audit history still work. Invalid values are rejected. Unauthorized users cannot update controls. Master, auto-warning and version semantics are unchanged, and all test values are restored after execution.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

CMR-UI-003 - Responsive, keyboard and error-state QA	P1
Preconditions / build / role	Desktop and mobile browser/device; keyboard-only input; browser zoom 100% and 200%; ability to simulate validation error and infrastructure-disabled state.
Test data	Viewports 390px, 768px, 900px and 1200px; long translated/admin font rendering where available.
Numbered steps	1. Resize through each viewport. 2. Tab through switches, input, Save and Recent changes. 3. Test at 200% zoom. 4. Trigger a validation error. 5. Inspect disabled controls.
Expected result	The controls remain one line on desktop and stack cleanly below 900px. Focus is visible, labels are announced, touch targets remain usable, errors remain visible, disabled controls cannot submit, and the evidence locator and call list retain their prior layout and behavior.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

Hima — Support Bot

AI ticket-deflection feature — **how it works**, configuration, and operations

Innovfix Pvt. Ltd. · Hima App · v1 · 2026-07-17 · Backend: Laravel (demo_hima / hima-admin-panel) · App: Android (hima_app)

> **What it is, in one line.** A guided in-app chat that sits **in front of** the existing ticket system: it asks the user 3–4 questions, reads **their own live account data**, answers, and only creates a support ticket if the user says the answer did not help. The goal is to stop the repeat-contact tickets (the average user raised ~2 per month) at the source — without ever making a false statement about someone's money or account.

1. The user flow

Step	Screen	Notes
0 · Language	"Choose your language" (prompt in English, all 11 options)	Applies to **this conversation only** — does not change the profile language / matching.
1 · Category	5 categories, volume-ordered	Calls · Earnings/Withdrawal · Coins/Recharge · Account/Profile · Something else.
2 · Sub-category	The category's sub-options + "My issue isn't listed"	Chips are **server-driven** — the app renders labels and posts keys; it knows nothing about the taxonomy.
3 · One detail	A sub-category-specific question (optional, with Skip where safe)	e.g. "double charge" asks for a date/time; the answer genuinely needs it.
4 · Answer	Typing → the AI's answer	Only this step is slow. The bot has already read the user's data before replying.
5 · Did this solve it?	[Yes, solved] · [No, still need help]	**Yes** → close, **no ticket**. **No** → one more attempt, then a human.

> **Yes vs No — the whole point.** "Yes" closes the chat and **creates no ticket row** (a deflected issue must not appear in `tickets`, or deflection is unmeasurable). "No" buys **one** second, different answer; a second "No" creates a ticket for a human, pre-tagged and with the bot's diagnostics attached. There is no third attempt.

2. How it stays accurate (why it can be trusted with money)

- **Reads live data via fixed, read-only tools** — never free-form SQL. Eight tools: account summary, recent calls, call detail, recharge history, coin ledger, withdrawal history, earnings summary, block status.
- **The user is structural, not a parameter.** The logged-in user id is fixed on the server; it appears in **no** tool the model can call, so the model cannot express "look at another user's data" (closes IDOR by construction).
- **Grounding guard.** Every number in a reply (₹ amounts, coins, order ids, times) must be a value a tool actually returned. Any invented number → the answer is discarded and the issue is escalated.
- **Skip-AI on money.** For sensitive/redirect money categories the model is never called at all — a human takes it directly, so money cannot be hallucinated about.
- **Recharge crediting is never asserted as fact.** `transactions` has no gateway/order link and `add_coins` also covers admin grants, so a same-size credit near a payment is only an **indication**, not proof. The bot states **payment_status** (paid / not completed) as fact, reports any nearby credit as a labelled possibility with its confidence, and routes every paid recharge to a human to verify against the gateway dashboard — it never claims "you were credited" or "money taken, no coins".
- **Fail closed.** If a tool cannot read the user's data (DB/replica down), the bot escalates to a human rather than answering around the gap. On prod, reads use a replica; if the replica lags, it falls back to the primary so it never states stale data.
- **Never promises** a refund, credit or timeline; **never deletes** an account — a delete request is treated as a symptom and probed.

3. Configuration & kill-switches (ops)

Setting	Where	Effect
`support_bot_enabled`	`gateway_config`	**Master kill-switch.** When set to `0`, the app shows the old raise-ticket form and in-flight sessions escalate to a human instead of calling the model. Fails **closed** (unreadable = off).
`support_bot_model`	`gateway_config`	The model id (default Claude Sonnet). Deliberately separate from the ticket-answering model.
`support_bot_subcategories`	`gateway_config`	CSV allowlist of sub-categories (empty = all). An off-list topic routes straight to a human — turn the bot on one topic at a time as shadow data proves each playbook.
`known_issue_banner`	`gateway_config`	When set, shown first during a platform incident — deflects the duplicate flood before the menu.
`support_hours_start` / `_end`	`gateway_config`	Agent shift (default 4 PM–12 AM IST) for the out-of-hours "when we'll look" ETA.
`SUPPORT_BOT_READ_CONNECTION`	.env	Read replica for tool queries on prod (empty on demo). Lag-guarded.
`SUPPORT_BOT_SESSION_CEILING`	.env	Hard per-session deadline (25s) — under PHP's execution limit; the app waits longer so it always gets the definitive answer.
`SUPPORT_BOT_TOKEN_BUDGET`	.env	Hard per-session cost ceiling; exceeding it escalates rather than spending more.

> **Answers are ops-editable without a deploy.** The per-sub-category playbooks live in `ai_reference_faqs` (already per-language, already an admin UI). The Yes/No taps are a free labelled dataset — a sub-category below ~40% "Yes" is a broken playbook or a real product bug.

4. Endpoints (all `auth:api`, no user_id ever accepted from the client)

Endpoint	Purpose
`support_bot_start`	Opens/*resumes* a session; returns language chips + any known-issue banner. Checks for a pending escalated ticket up front.
`support_bot_reply`	Walks the tree, then answers. One request per session at a time (locked).
`support_bot_feedback`	"Did this solve it?" — the only thing that resolves or escalates. Runs the second attempt.
`support_bot_session`	Rebuilds a session reopened after the app was killed (transcript + current step).
`support_bot_attach`	Optional file on the ticket the bot raised (image/video/audio, ≤5MB). Serialised per ticket.

5. Rollout & what to watch

- **Demo-first, always.** Everything is verified on `demohima.himaapp.in` before prod. Prod goes on an explicit go, staged by sub-category allowlist and user %.
- **Fallback is safe.** With the kill-switch off, or on any start failure, the user gets the old form — the support channel is never removed behind the LLM.
- **Weekly metrics:** per-sub-category Yes-rate · grounding-guard trips (should be ~0) · p50/p95 latency · tokens/session · **48h re-contact rate** · and the north star: **tickets per user per month.**

> **Design guardrail — "calls not coming" (the #1 driver) is a real product bug the bot cannot fix.** The bot reads the data and escalates; it does **not** troubleshoot ("check your network"), which reads as being fobbed off by a robot. Its deflection target there is near-zero **by design** — track it as a product-bug signal, not a deflection KPI.

6. Technical inventory

Controllers, services & models

Component	Type	Responsibility
`SupportBotController`	Controller	The 5 endpoints; step machine; session lock; kill-switch guard; escalation + attach.
`SupportBotService`	Service	The model tool-loop; deadline budget; grounding guard; escalate control-flow.
`SupportToolRunner`	Service	The 8 read-only tools; user-scoped by constructor; replica-lag guard; recharge matcher.
`TicketCreationService`	Service	Turns a session into a human ticket; per-user advisory lock; canonical-phrase routing.
`SupportBotSafetyGate`	Service	Safety-only refusal (harassment/abuse/minors/self-harm/report).
`SupportHours · SupportBotStrings · SupportBotTaxonomy · Prompts\SupportBotPrompt`	Service	Shift/out-of-hours ETA · 11-language strings · categories & canonical phrases · the versioned system prompt.
`SupportBotSession`	Model	Per-conversation state (step, language, transcript, last_render, telemetry).
`Ticket · Users`	Model	Existing models; bot writes tickets with `source=bot`; reads Users for scope.

Tables (reads unless noted)

Table	Use
`support_bot_sessions` (RW)	Session state + telemetry. New table; migrations are `hasColumn`-guarded.
`tickets` (RW)	Human tickets on escalation (bot columns: source, sub_category, language, bot_session_id, screenshot).
`gateway_config`	Kill-switch, model id, known-issue banner, support hours. Runtime, cross-node.
`users · user_calls · transactions · coins · cashfree_payments · phonepe_payments · blocked_users`	Read-only tool sources (account, calls, ledger, packs, payments, block status).
`ai_reference_faqs`	Per-language answer playbooks (admin-editable, no deploy).

External providers · jobs

- **OpenRouter → Claude Sonnet** (via `gateway_config.support_bot_model`) — the only external call; synchronous, budgeted (25s ceiling, token cap), keyed by `openrouter_api_key`.
- **Cashfree / PhonePe** — **read-only**, via their payment tables; the bot never calls the gateways.
- **Jobs/cron:** the bot adds **none** — every request is synchronous. Escalated tickets flow through the existing ticket pipeline. No queue worker to run or monitor.

Failure modes & handling (all fail toward a human, never a wrong answer)

Failure	Handling
Model timeout / API error / round limit	Escalate to a human ticket (deadline budget → `escalate`).
Tool can't read data (DB/replica down)	Fail closed — escalate (`tool_error`), never answer around the gap.
Replica lagging	Read from the primary for that session.
Ungrounded number in the reply	Discard the answer, escalate (grounding guard).
Recharge crediting unprovable	Report payment_status only; route to a human to verify (no credited/no-coins claim).
Kill-switch off (start or mid-session)	Old raise-ticket form / escalate the in-flight session; no model call.
Duplicate ticket race / concurrent attach	Per-user & per-ticket MySQL advisory locks (cross-node).
Node-local attachment storage	**Known operational risk** — an upload may be unavailable from another node until ticket storage is moved to a shared disk (platform task).

Rollback

- **Instant:** set `gateway_config.support_bot_enabled=0` — users get the old form, in-flight sessions escalate. No deploy, cross-node.

- **All users, staged by topic:** the bot is on for every user when enabled (no per-user gating, by owner decision). Turn it on one topic at a time with `support_bot_subcategories` (a `gateway_config` allowlist; empty = all topics) — off-list topics route straight to a human.
- **Schema:** all new columns are additive and `hasColumn` -guarded; reverting the branch leaves the tables harmless. No destructive migration.

7. QA test cases (formal — run before release)

> **How to run.** Each case has a stable ID (`SB-###`), priority (P1 blocks release), module reference, preconditions and test data, numbered steps, and the expected result. Fill the **Execution** row: mark Pass/Fail, attach Evidence (screen recording / DB row), log a Defect ID on failure, and record the Retest result. **Build/role/data:** run on the Play/Internal build, as a **male caller** and a **female creator**, on WiFi **and** mobile data, unless a case says otherwise.

SB-001 P1 **Guided flow, end to end** Module: Support Bot / Flow · Ref §1

Precondition	Bot enabled (`support_bot_enabled=1`); test account has no open escalated ticket.
Test data	Any account; pick category "Calls".
Steps	1. Open Help → Support. 2. Observe the language prompt; select one. 3. Select a category, then a sub-category. 4. Answer the follow-up (or Skip if offered). 5. Wait for the answer, then observe the "Did this solve it?" controls.
Expected	Steps appear in order language → category → sub-category → optional detail → answer → Yes/No. "My issue isn't listed" is present at the sub-category step. No crash; each step responds <2s except the answer.

Execution: ☐ Pass ☐ Fail **Evidence:** ___________ **Defect#:** _____ **Retest:** _____

SB-002 P1 **Language & script consistency** Module: Support Bot / Language · Ref §1.0

Test data	Select English at step 0; then type the free-text detail in Tanglish ("calls varala").
Steps	1. Select a language and complete the flow to an answer. 2. Read every bot message for language consistency. 3. At the free-text step type romanised (Tanglish) text.
Expected	The whole conversation stays in the chosen language (no mid-chat switch). The answer mirrors the user's script (Tanglish in → Tanglish out), not native Tamil.

Execution: ☐ Pass ☐ Fail **Evidence:** ___________ **Defect#:** _____ **Retest:** _____

SB-003 P1 **Deflection creates no ticket** Module: Support Bot / Deflection · Ref §1.5

Precondition	DB/admin access to check the `tickets` table.
Steps	1. Complete a flow to an answer. 2. Tap "Yes, solved". 3. Query `tickets` for this user/timestamp.
Expected	Conversation closes; optional rating shows; **no new row** in `tickets` .

Execution: ☐ Pass ☐ Fail **Evidence:** ___________ **Defect#:** _____ **Retest:** _____

SB-004 P1 **Escalation: second attempt then human** Module: Support Bot / Escalation · Ref §1.5

Steps	1. Reach an answer; tap "No, still need help". 2. Observe the second attempt (a different answer). 3. Tap "No" again. 4. Open the ticket in admin.
Expected	Exactly one second attempt (different content). A ticket is created for a human (`source=bot` , `requires_manual_support=1`) with the bot's diagnostics attached. No third AI attempt.

Execution: ☐ Pass ☐ Fail **Evidence:** ___________ **Defect#:** _____ **Retest:** _____

SB-005 P1 **Delete-account is probed, never actioned** Module: Support Bot / Safety · Ref §2

Steps

1. Choose Account → a delete-account intent (or type "delete my account").
2. Follow the conversation.

Expected The bot asks what the underlying problem is and tries to help; it never offers or performs deletion. No account is deleted or disabled.

Execution: □ Pass □ Fail **Evidence:** ___________ **Defect#:** _____ **Retest:** _____

SB-006 P1 **Resume mid-flow after app kill** Module: Support Bot / Resume · Ref §4

Steps

1. Advance to the sub-category / detail step.
2. Force-kill the app (swipe from recents).
3. Reopen Help → Support.

Expected The prior conversation is rebuilt and lands on the same step; the user does not re-answer earlier questions.

Execution: □ Pass □ Fail **Evidence:** ___________ **Defect#:** _____ **Retest:** _____

SB-007 P2 **Resume after answer / when finished** Module: Support Bot / Resume · Ref §4

Steps

1. Receive an answer (Yes/No shown); kill and reopen.
2. Resolve or escalate the session; kill and reopen again.

Expected First reopen shows THAT answer with Yes/No (never the rejected first answer). After resolve/escalate, reopen is read-only (transcript only, no live input).

Execution: □ Pass □ Fail **Evidence:** ___________ **Defect#:** _____ **Retest:** _____

SB-008 P2 **Attachment — validation & lifecycle** Module: Support Bot / Attach · Ref §4

Precondition A ticket has just been raised (SB-004), so the attach option is offered.

Test data A <5MB image; a >5MB video; a .txt file; a large screen-recording.

Steps

1. Attach the valid image → confirm it appears on the ticket.
2. Attach the >5MB file → observe rejection.
3. Attach the .txt → observe type rejection.
4. Start a large upload, then rotate the screen; and separately, start one and background/kill the app.
5. Try to pick a second file while one is uploading.

Expected Valid file uploads (visible on the ticket in admin). Too-large/wrong-type are rejected with a clear message and are NOT shown as accepted. Rotation mid-upload does not break the upload or crash. Killing mid-upload leaves no orphaned cache file. A second pick while uploading is blocked with a "wait" message.

Execution: □ Pass □ Fail **Evidence:** ___________ **Defect#:** _____ **Retest:** _____

SB-009 P2 **Slow-network patience** Module: Support Bot / Timeout · Ref §1.4

Test data Throttle the network (e.g. slow 3G profile).

Steps

1. Reach the answer step on a slow connection.
2. Repeat for the second attempt (after "No").

Expected The app waits (up to ~30s) for the definitive answer or escalation on BOTH the first and second attempts; it does not fail early or dump the user to the old form.

Execution: □ Pass □ Fail **Evidence:** ___________ **Defect#:** _____ **Retest:** _____

SB-010 P1 **Kill-switch (start & mid-session)** Module: Support Bot / Ops · Ref §3

Precondition Ability to set `gateway_config.support_bot_enabled`.

Steps

1. Set the flag to 0; open Help → Support.
2. Set it back to 1; start a session and reach the sub-category step.
3. Set it to 0 mid-session; send the next step.

Expected With the flag 0, the old raise-ticket form is shown (no bot). Flipping to 0 mid-session makes the next step escalate to a human (no AI call), with no hang or crash.

Execution: □ Pass □ Fail **Evidence:** ___________ **Defect#:** _____ **Retest:** _____

SB-011 P2 **Pending ticket & duplicate guard** Module: Support Bot / Tickets · Ref §1

Precondition	The test account already has one open escalated ticket.
Steps	1. Open Help → Support. 2. Separately, from two devices, drive one session each to escalation at the same time.
Expected	The user is told about the open ticket at the START (not after four questions). No duplicate ticket is created, even from two concurrent devices.

Execution: ☐ Pass ☐ Fail **Evidence:** ___________ **Defect#:** _____ **Retest:** _____

SB-012 P1 **MONEY — recharge is verified, never invented** Module: Support Bot / Money · Ref §2 · MUST-PASS

Precondition	DB access; a creator and a male account. Verify every figure against the DB, never the reply alone.
Test data	A real ₹1–₹49 recharge; also one completed >30 min after starting the order.
Steps	1. Ask a recharge/coins question after a real recharge. 2. Compare the bot/ticket diagnostics to the `cashfree_payments` / `phonepe_payments` and `transactions` rows. 3. Repeat for a delayed (>30 min) completion and for two close same-size purchases.
Expected	Recharge routes to a human. Diagnostics state **payment_status** as fact only; crediting is shown as INDICATIVE (never a definitive "you were credited" or "money taken, no coins"). A delayed credit is still matched; two close same-size purchases with one credit are marked for human review, not falsely credited. Every ₹/coin/order-id/time in any reply came from the data (no invented refund/timeline).

Execution: ☐ Pass ☐ Fail **Evidence:** ___________ **Defect#:** _____ **Retest:** _____

SB-013 P1 **MONEY — earnings connected count** Module: Support Bot / Money · Ref §2 · MUST-PASS

Precondition	A creator account with some calls that never connected (`started_time='00:00:00'`).
Steps	1. Ask an earnings question. 2. Compare the "connected calls" figure to the Recent Calls tool and the DB.
Expected	The connected count excludes `00:00:00` (never-connected) calls and matches Recent Calls exactly.

Execution: ☐ Pass ☐ Fail **Evidence:** ___________ **Defect#:** _____ **Retest:** _____

SB-014 P1 **SECURITY — own data only (IDOR)** Module: Support Bot / Security · Ref §4 · MUST-PASS

Precondition	Two accounts A and B; a tool to send raw API requests (Postman).
Steps	1. As A, drive a bot session; confirm it only ever reflects A's data. 2. With A's token, call the ticket routes passing B's user_id / mobile number.
Expected	The bot only ever shows A's own data. Ticket create/list use the token identity — passing B's user_id/mobile returns A's own tickets or 401, never B's data; unregistered vs registered numbers return identical responses (no enumeration).

Execution: ☐ Pass ☐ Fail **Evidence:** ___________ **Defect#:** _____ **Retest:** _____

SB-015 P3 **Zoho launcher hygiene** Module: Support Bot / UI · Ref §5

Steps	1. As a female account (launcher enabled from home), open Support and leave. 2. As a male account, open Support and leave.
Expected	Female: the floating Zoho launcher is hidden on the bot screen and restored on exit. Male: the launcher never appears.

Execution: ☐ Pass ☐ Fail **Evidence:** ___________ **Defect#:** _____ **Retest:** _____

Support Bot Selected-Language Authority

Bug fix SBLANG-2026-07-20. These MUST-PASS cases supersede SB-002 wherever SB-002 says typed text can change the reply language. The chatbot selection now always wins.

SB-016 | P1 MUST-PASS | Original reproduction and multi-turn regression

Module / ref	Support Bot / Language / Multi-turn - SBLANG-2026-07-20 - prompt v10
Build / environment	Play/Internal Android build connected to demo backend with the support bot enabled. Run on WiFi and mobile data.
Account role	Female creator whose app profile language is Tamil. Repeat once with a male caller.
Preconditions	No open escalated ticket. Start a new bot session. Do not reuse the affected terminal session from 2026-07-20.
Test data	Inside the chatbot select Kannada. Choose Calls & Connection, then Call auto-cut / disconnects. Enter Last user. When asked for a time, answer in English or Tamil text.
Steps	1. Open Help and Support and start a new chatbot session. 2. Select Kannada even though the account profile is Tamil. 3. Complete the Calls & Connection / disconnect path using the test data above. 4. Read the category prompt, detail prompt, AI clarification and final AI answer. 5. Reply to the clarification in English or Tamil and continue to the final answer. 6. Background and reopen the app; verify the restored transcript and active input state. 7. Tap No once and inspect the second AI attempt. 8. Capture screen recording plus the redacted session language, prompt_version and step from admin/DB evidence.
Expected	- Every user-visible bot line after selection is Kannada. The Tamil profile, English chip label and Tamil/English free text never switch the reply language. - Clarification uses a text box, the reply is accepted, and the final/second answer remains Kannada. - The restored transcript does not re-render a Tamil answer or reopen the wrong step. - The session stores language=Kannada and prompt_version=v10-20260720. No users.language update occurs. - No duplicate provider request, duplicate answer or duplicate ticket occurs on background/resume.
Execution	[ ] Pass [ ] Fail Evidence: ____________________ Defect ID: __________ Retest: __________

Release rule

A single Tamil, English or other-language AI bubble after Kannada is selected is a release blocker. Do not accept a correct script with the wrong spoken language (for example Romanized Tamil).

Support Bot Selected-Language Authority

Cross-language matrix and fail-closed behavior. Provider output must be stubbed for the negative checks; do not spend or transmit real user data merely to force a bad model response.

SB-017 | P1 MUST-PASS | All-language matrix and wrong-language fail-safe

Module / ref	Support Bot / Language Guard - SBLANG-2026-07-20 - all 12 chatbot languages
Build / environment	Backend unit/integration environment with a fake OpenRouter response, plus one normal demo smoke per selected language. No live provider fault injection.
Account role	At least two test accounts. Pair each selected language with a different profile language.
Test matrix	English, Tamil, Hindi, Telugu, Kannada, Malayalam, Bengali, Marathi, Gujarati, Odia, Punjabi and Assamese. Test native-script and Romanized output for the 11 Indian languages; test clear standard English in Latin script for English.
Steps	1. For each language, set the account profile to a different language and select the target only inside the chatbot. 2. Complete one normal category and a multi-turn clarification; verify fixed strings and AI text stay in the selected language. 3. In the fake-provider test, return a confident different Romanized language (including the reported Tamil response for selected Kannada). 4. Verify one corrective rewrite requests only the selected language in a verifiable script: standard English/Latin for English and native script for the 11 Indian languages. 5. Make the fake provider return the wrong language again and observe the terminal behavior. 6. For selected English, inject confident Romanized Tamil and verify it is blocked; then return clear English and verify it passes. 7. Inspect the model-visible account summary and tool schema for absence of users.language. 8. Repeat with an unsupported/empty session language and verify no provider/tool call begins. 9. Record request count, response time, ticket count and sanitized language-guard logs.
Expected	- The selected chat language is the sole authority in all 12 rows; profile, typed text, FAQ/tool content and prior turns cannot override it. - English is a guarded support-chat choice: clear English passes, while confidently detected Romanized Indian-language output is rejected. - A valid selected-language answer passes without an extra provider call. - A wrong or unverifiable answer gets at most one bounded rewrite. The wrong text is never displayed. - If the rewrite is still wrong, the flow fails closed to one human ticket and uses fixed translated scaffolding in the selected language. - An invalid session language fails before tool/provider work. No profile-language fallback occurs. - Normal API/query count is unchanged; guard CPU remains negligible and no unbounded retries are possible.
Regression	Re-run SB-001, SB-004, SB-005, SB-006, SB-007, SB-009, SB-010, SB-011 and SB-014. Confirm grounding, timeout, idempotency, resume, ticket and kill-switch behavior are unchanged.
Execution	[ ] Pass [ ] Fail Evidence: ____________________ Defect ID: __________ Retest: __________

NET-009 Server-Lifetime Ceiling - Shadow Mode

SOP revision 1.5 - 2026-07-21 - Query-free old-versus-proposed comparison. The shadow flag may write END_CLAMP_LOG records only; it must not alter duration, coins, creator income, history or responses.

Release gate	CBS-001 through CBS-007 are MUST-PASS before any production shadow activation. CBS-008 and CBS-009 are MUST-PASS before Finance may consider staged enforcement. Production deployment, flag changes, refunds and enforcement each require separate owner approval. Record redacted evidence and exact node/build/hash.

CBS-001 - Default-off and strict no-money-change boundary

MUST-PASS / P0

Preconditions / build / role	QA backend containing NET-009; shadow flag absent; caller and creator accounts with recorded starting wallets; audio/video direct/random test calls.
Test data	Calls of 5, 18, 59, 60, 61 and 600 seconds; capture API response, user_calls, both wallets and call-linked ledgers.
Numbered steps	1. Record baseline balances and call row. 2. Complete each call through update_connected_call and individual_update_connected_call. 3. Confirm no server_lifetime_v1 log is written while the flag is absent. 4. Reconcile every financial and visible field against the unpatched build.
Expected result	Existing billing behavior is byte-for-byte equivalent at the API/data level. No proposed duration reaches caller debit, creator credit, call row, transactions, Recent Calls or notifications.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

CBS-002 - Shadow flag logs but never enforces

MUST-PASS / P0

Preconditions / build / role	QA backend; server_lifetime_ceiling_shadow.flag present; same accounts and test data as CBS-001; log access without customer PII export.
Test data	One normal call and one fast-clock call for each of the two normal settlement routes.
Numbered steps	1. Record wallets and expected existing charge. 2. Enable the shadow flag. 3. Settle calls. 4. Find exactly one valid server_lifetime_v1 END_CLAMP_LOG per winning settlement. 5. Compare response, row, wallets and ledgers with current calculation. 6. Remove flag and repeat one call.
Expected result	Logs contain existing/proposed seconds, rounded/billed minutes, coins and delta. Live financial and user-visible results remain current behavior. Removing the flag immediately stops new shadow records.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

CBS-003 - Replay the four production-proven overbilling incidents

MUST-PASS / P0

Preconditions / build / role	Offline fixture or isolated QA data only; no production replay/write; shadow flag present; video rate 60 coins/minute.
Test data	Server lifetimes/current results: 5s/59:20/3600, 36s/59:50/1500, 19s/59:44/3600, 18s/3:25/240.
Numbered steps	1. Replay each normalized timestamp set through the pure calculator and matching endpoint harness. 2. Capture END_CLAMP_LOG. 3. Verify current billing variables remain unchanged in shadow mode. 4. Sum proposed coins and would-save coins.
Expected result	Proposed durations are 5, 36, 19 and 18 seconds; proposed charges are 0, 60, 60 and 60 coins; total proposed maximum is 180 and minimum would-save is 8,760 coins. Shadow mode still moves no money.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

CBS-004 - NET-007 genuine reconnect recovery is preserved	MUST-PASS / P0
Preconditions / build / role	QA backend; shadow flag present; controllable reconnect; current Android timer reset behavior represented.
Test data	Call created 10:00, reconnect reset submitted start to 10:20, end 10:21, settlement received 10:21; audio and video.
Numbered steps	1. Settle through both normal endpoints. 2. Prove NET-007 applied. 3. Inspect shadow comparison. 4. Compare current and proposed duration/coins. 5. Repeat with a delayed settlement received after 10:21.
Expected result	Existing and proposed duration both remain 1,260 seconds. Rounded billing and creator treatment are unchanged. The ceiling never shortens recovered talk time when it fits inside server lifetime.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

CBS-005 - Delayed settlement cannot enlarge billing	MUST-PASS / P0
Preconditions / build / role	QA backend; shadow flag present; end request delayed using controlled network/worker scheduling.
Test data	One-minute client duration, call row created at 10:00, request delivered at 10:07; direct/random and audio/video.
Numbered steps	1. Complete a one-minute call. 2. Hold the end request for five minutes. 3. Deliver it. 4. Inspect server lifetime, proposed duration and old/new coins. 5. Reconcile stored/live result.
Expected result	Server lifetime may be seven minutes, but proposed duration remains one minute because MIN never increases client duration. Shadow mode makes no financial change.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

CBS-006 - Midnight, invalid time and fail-open boundaries	MUST-PASS / P0
Preconditions / build / role	QA backend; shadow flag present; timezone fixed to the production Laravel timezone; injectable malformed/future call datetime and logging failure.
Test data	23:59 to 00:01 overnight call; ended_time earlier than started_time without overnight threshold; call.datetime after receipt; invalid call.datetime; logger exception.
Numbered steps	1. Run the legitimate overnight call through existing normalization. 2. Run each invalid boundary. 3. Inspect current settlement result and shadow log/error handling. 4. Confirm no exception escapes the helper.
Expected result	The legitimate overnight duration remains two minutes. The helper uses full server datetimes, never performs an absolute negative conversion, reports invalid input safely, and logging/calculation failure cannot interrupt existing settlement.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

CBS-007 - All three financial routes are covered	MUST-PASS / P0
Preconditions / build / role	QA route list and controller; shadow flag present; valid JWT; isolated accounts; endpoint access logs.
Test data	update_connected_call, individual_update_connected_call and every_min_update_connected_call; one normal and one over-ceiling request per route.
Numbered steps	1. Invoke each route using valid route-specific state. 2. Verify one server_lifetime_v1 log for each successful financial execution. 3. Confirm every_min retains its current legacy affordability/rate semantics. 4. Verify duplicate/already-settled normal requests do not create a false winning-settlement record.
Expected result	No settlement method bypasses comparison. The shadow helper adds no database query and does not repair or worsen the legacy route's existing non-idempotency; all live financial behavior remains unchanged.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

CBS-008 - One-day three-node shadow reconciliation and capacity gate	MUST-PASS / P0 / SHADOW
Preconditions / build / role	Separate approval for deployment and flag activation; identical reviewed files on app-1/app-2/app-3; enforcement remains disabled; disk/latency/error monitoring available.
Test data	One complete day of production settlements, grouped by route/type/mode and duration delta; redacted call IDs only for bounded investigation.
Numbered steps	1. Verify hashes and default-off state on all nodes. 2. Activate shadow only after approval. 3. Monitor log bytes, disk headroom, PHP CPU/RSS, endpoint p50/p95/p99, 5xx and log failures. 4. Reconcile counts with successful settlement counts. 5. Compare normal versus changed buckets. 6. Remove shadow flag after the approved window.
Expected result	Normal calls have zero proposed billing difference; only server-envelope anomalies change. No wallet/ledger/history delta is caused by NET-009. No material latency, CPU, memory, disk or error regression occurs, and logs stop when the flag is removed.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

CBS-009 - Rollback and Finance enforcement gate	MUST-PASS / P0 / APPROVAL
Preconditions / build / role	Completed CBS-001 through CBS-008 evidence; Finance/Product review; reviewed rollback package; no enforcement code or flag active.
Test data	Changed calls segmented by reconnect evidence, ring/setup envelope, route, call type, connection type and delta bucket.
Numbered steps	1. Review one-day reconciliation and false-positive samples. 2. Confirm normal-call delta is zero. 3. Confirm four incidents and reconnect fixtures. 4. Record Finance decision on ring/reconnect tradeoff and refunds. 5. Verify rollback removes shadow flag/logging without touching money. 6. Do not enable enforcement in this release.
Expected result	A signed decision records whether staged enforcement may be built. Shadow deployment can be rolled back independently. No production money behavior changes until a separate reviewed enforcement patch, staged flag approval and smoke plan exist.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

RELEASE TEST APPENDIX MCT

Midnight Call Remaining-Time Hotfix

Production backend hotfix - 2026-07-22 - live hash 2c1099563c4d3e781d77dc22aa3520c3322da1e9dd24aa8def5e236656c99368. Scope: date-aware elapsed-time calculation in get_remaining_time, the enforced switch-session calculation, and post-call duration display. No Android, schema or settlement formula change.

Release gate	MCT-001 through MCT-010 are MUST-PASS. Test audio and video with funded caller and creator accounts. Capture redacted call IDs, API remaining_time, UI timer, disconnect behavior, user_calls, linked transactions, creator income, history and all-three-node evidence. The separate duplicate switch-leg billing defect is not fixed by this hotfix.

MCT-001 - Daytime audio/video behavior is unchanged

MUST-PASS / P0

Preconditions / build / role	Production or production-equivalent backend with the deployed hash; current Play Android build; funded caller and creator; audio and video enabled.
Test data	One 10-minute daytime audio call and one 10-minute daytime video call; record balance and expected full-minute allowance.
Numbered steps	1. Record starting coins. 2. Connect each call away from midnight. 3. Capture get_remaining_time at connection and at two later refreshes. 4. End normally. 5. Reconcile call row, caller debit, creator income and history.
Expected result	Remaining time decreases by real elapsed seconds with the existing response format. No false zero, timer jump, disconnect, billing, income or history regression occurs.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

MCT-002 - Funded call continues across midnight

MUST-PASS / P0

Preconditions / build / role	Current Play build; caller funded for at least 30 minutes; creator available; synchronized screen recording and API/log evidence.
Test data	Audio and video calls connected between 23:50 and 23:55 IST and kept connected until at least 00:05 IST.
Numbered steps	1. Record wallets and call row. 2. Connect before midnight. 3. Observe the timer across 00:00 and two API resync intervals. 4. Keep the call active after 00:05. 5. End normally and reconcile data.
Expected result	Elapsed time crosses midnight normally; remaining_time stays positive and decreases continuously. No false insufficient-coins popup or automatic midnight disconnect occurs.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

MCT-003 - Dial before midnight and answer after midnight

MUST-PASS / P0

Preconditions / build / role	Controlled QA accounts and a call that can ring across midnight; caller funded; server and device times recorded.
Test data	Create/ring at 23:59:55-23:59:59; answer at 00:00:01-00:00:10; test audio and video.
Numbered steps	1. Start ringing just before midnight. 2. Answer just after midnight. 3. Capture the stored datetime and started_time. 4. Observe three remaining-time refreshes. 5. End after at least two minutes.
Expected result	The connected start resolves to the following day, not nearly 24 hours earlier. Timer, disconnect behavior, duration and billing reflect only actual connected time.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

MCT-004 - Audio/video switch keeps session elapsed time across midnight	MUST-PASS / P0
Preconditions / build / role	switch_balance_guard=enforce; funded caller; switch-capable current Play build; database/log access with redacted identifiers.
Test data	Start audio before midnight and switch to video after midnight; repeat video-to-audio. Include a switch immediately before 00:00.
Numbered steps	1. Record root leg datetime/started_time. 2. Cross midnight and switch call type. 3. Capture SWITCH_BALANCE_GUARD and API values. 4. Verify the new leg does not reset the full allowance. 5. End and reconcile all legs.
Expected result	The guard uses the root leg's real date and whole-session elapsed time. Remaining time stays positive when funded, does not reset, and does not become a false zero at midnight.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

MCT-005 - Real insufficient balance still disconnects	MUST-PASS / P0
Preconditions / build / role	Caller funded for exactly one full minute at the applicable audio/video conversion rate; creator available; current Play build.
Test data	One daytime call and one call whose final paid minute ends just after midnight.
Numbered steps	1. Record exact starting coins. 2. Connect and do not recharge. 3. Observe countdown to zero. 4. Wait through the app confirmation/resync interval. 5. Reconcile the settled call and wallets.
Expected result	Only genuine exhaustion returns zero and triggers the existing insufficient-coins flow. The hotfix does not grant free time or prevent the intended disconnect.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

MCT-006 - Post-call duration is correct across midnight	MUST-PASS / P0
Preconditions / build / role	Caller/creator accounts; rating or post-call prompt enabled; audio and video calls that cross midnight.
Test data	23:52:00 to 00:07:00 (expected 900 seconds) plus a daytime 15-minute control call.
Numbered steps	1. Complete both calls. 2. Open the post-call prompt. 3. Capture user_call_duration response and displayed duration. 4. Compare with stored start/end timestamps and control call.
Expected result	The overnight call reports 900 seconds / 00:15:00, not approximately 23 hours. The daytime control remains unchanged.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

MCT-007 - History and settlement remain correct	MUST-PASS / P0
Preconditions / build / role	Funded caller/creator; access to user_calls and call-linked transaction rows; no manual wallet adjustment.
Test data	Completed audio/video midnight calls from MCT-002 and MCT-003.
Numbered steps	1. Wait for normal settlement. 2. Confirm one completed row per actual leg. 3. Open Recent Calls on both roles. 4. Reconcile elapsed duration, rounded billed minutes, debit and creator income. 5. Retry history refresh.
Expected result	Calls remain visible and are billed through their actual recorded end using existing settlement rules. The timer hotfix creates no missing history, free segment or extra financial row.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

MCT-008 - Clock skew and signed-duration boundaries — MUST-PASS / P0

Preconditions / build / role	Production-equivalent deterministic harness using the same Carbon version as PHP 8.2 and 8.4.
Test data	Daytime 600s; cross-midnight 900s; dial-before/answer-after 60s; one-second negative device skew; five-second future start.
Numbered steps	1. Execute every fixture on PHP 8.2 and 8.4. 2. Record normalized start and elapsed seconds. 3. Repeat 100 times. 4. Confirm no absolute negative conversion.
Expected result	Results are 600, 900, 60 and 61 seconds; a future start clamps to zero. Results are deterministic on both runtimes and never add an accidental day for small skew.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

MCT-009 - API compatibility, latency and failure regression — MUST-PASS / P1

Preconditions / build / role	Old and current supported Android builds; representative normal and switched calls; endpoint timing and query visibility.
Test data	At least 100 get_remaining_time samples per build/type plus baseline samples from the previous controller.
Numbered steps	1. Compare status code, JSON keys and value formats. 2. Measure p50/p95/p99. 3. Confirm normal calls add no query. 4. Confirm switched calls retain one bounded indexed root lookup selecting datetime. 5. Check 4xx/5xx and logs.
Expected result	Android compatibility is unchanged; no material latency or error regression occurs. Normal-call query count is unchanged and switched-call lookup remains bounded/indexed.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

MCT-010 - Three-node production smoke, first midnight and rollback readiness — MUST-PASS / P0 / PROD

Preconditions / build / role	Explicit production approval; identical live hash on app-1/app-2/app-3; backup hash ed5dd254a9d91cd0a4dc56c0cc643887437ae549c1f5dac3f02a8f7d58584269; monitoring owner assigned.
Test data	Node-local unauthorized endpoint smoke plus the first real midnight cohort; aggregate/redacted evidence only.
Numbered steps	1. Verify live and backup hashes on all nodes. 2. Verify PHP-FPM 8.2/8.4 active and no new errors. 3. Confirm unauthenticated endpoint returns 401 on each node. 4. Monitor midnight false-zero/disconnect rate, duration, billing and 5xx. 5. Keep rollback command and owner ready.
Expected result	All nodes remain healthy and identical. Funded cross-midnight calls continue; genuine zero-balance behavior remains. Any new 5xx, widespread timer failure or billing anomaly triggers coordinated restore from the saved backup and graceful reload.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

RELEASE TEST APPENDIX SDG

Switch Duplicate Creation Guard - Log Mode

Production backend observation patch - 2026-07-23 - live controller hash 8c9b5f8eb14073aebfa32ff21c54bfa116a838110077bf3e853fe2c653711cda. The storage flag enables one bounded detector query and warning log only. It does not reuse a row, block a call, change a response, settle a call, deduct coins, credit income or refund history.

Release gate	SDG-001 through SDG-009 are MUST-PASS before log-mode evidence is used to design enforcement. Log mode is not the duplicate-billing fix: confirmed duplicate fixtures must still behave exactly as before while producing SWITCH_DUP_CREATE_GUARD_WOULD_REUSE. Enforcement, settlement blocking and refunds require separate approval.

SDG-001 - Flag absent preserves the previous implementation — MUST-PASS / P0

Preconditions / build / role	Production-equivalent backend with the new controller; switch_duplicate_creation_guard.log absent; funded caller/creator; audio and video switching available.
Test data	Normal calls, switch retries within 45 seconds, and one switch retry after 45 seconds.
Numbered steps	1. Confirm the flag is absent. 2. Run all calls. 3. Compare API responses, call rows, wallets, creator income, ledgers and logs with the previous controller. 4. Measure query count.
Expected result	No SWITCH_DUP_CREATE_GUARD log appears, no new detector query runs, and behavior/data are unchanged except for the inert filesystem check.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

SDG-002 - Confirmed delayed duplicate is observed but not blocked — MUST-PASS / P0

Preconditions / build / role	Isolated QA data or offline fixture only; log flag present; no production duplicate creation; both settlement routes represented.
Test data	Replay the 14 strict seven-day duplicate pairs, including creation gaps of 46 seconds and 251 seconds and near-identical video start/end times.
Numbered steps	1. Create the first open switch leg. 2. Submit the same-pair/type switch after the fixture delay. 3. Capture the response and warning. 4. Settle both rows in isolation. 5. Reconcile all fields.
Expected result	Every fixture writes one WOULD_REUSE warning with new/prior call IDs and mode=log. The second row is still created and current billing remains unchanged; log mode moves no money.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

SDG-003 - Existing 45-second retry dedupe remains authoritative — MUST-PASS / P0

Preconditions / build / role	Log flag present; controlled same-pair/type switch request; current call-create routes.
Test data	Repeat at 0, 10, 44 and 45 seconds; test both API routes and audio/video.
Numbered steps	1. Submit the initial request. 2. Retry at each boundary. 3. Record returned call ID, rows and logs. 4. Confirm no second settlement exists.
Expected result	The existing CALL_DEDUP_HIT path returns the original row. The new older-than-45-second detector does not add noise or change the boundary.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

SDG-004 - Legitimate sequential switches are not classified	MUST-PASS / P0
Preconditions / build / role	Log flag present; caller funded; normal audio/video handoff and settlement available.
Test data	Audio to video to audio to video, with each prior same-type leg ended before returning to that type.
Numbered steps	1. Complete every handoff sequentially. 2. Verify the prior leg is ended before the next creation. 3. Inspect rows and guard logs. 4. Reconcile timer and billing.
Expected result	No WOULD_REUSE warning appears for a completed prior same-type leg. Every legitimate sequential leg is created and billed under existing rules.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

SDG-005 - Pair, type, state and two-hour boundaries	MUST-PASS / P0
Preconditions / build / role	Log flag present; controlled rows for different creator, payer, type, connection type, ended state and age.
Test data	Open same pair/type at 46 seconds and 1:59:59; ended row; different type; different party; random call; row older than two hours.
Numbered steps	1. Submit a switch create for each fixture. 2. Capture warning/no-warning. 3. Confirm every API response and row outcome. 4. Check query plan.
Expected result	Only the open individual same-payer/creator/type row between 46 seconds and two hours is logged. Other fixtures are excluded without behavior change.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

SDG-006 - Both switch creation routes and old clients remain compatible	MUST-PASS / P0
Preconditions / build / role	Current and older supported Android builds; call_female_user and call_male_user; requests without channelName, matching current app contract.
Test data	Male-initiated and female-initiated switches, audio and video, with/without an older open candidate.
Numbered steps	1. Run each route/build combination. 2. Compare status, JSON keys and call IDs with the previous controller. 3. Inspect route-specific site values in logs.
Expected result	Responses remain compatible. Missing channelName does not fail creation. Candidate logs identify the correct route without recording user IDs or channel values.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

SDG-007 - Detector and logger failure are fail-open	MUST-PASS / P0
Preconditions / build / role	QA backend with injectable database read failure and logging failure; log flag present.
Test data	One normal switch and one delayed duplicate candidate for each failure mode.
Numbered steps	1. Force the detector query to throw. 2. Repeat with warning logging failure. 3. Capture API response, row and financial outcome. 4. Restore dependencies and retest.
Expected result	No detector/logger exception escapes the transaction. Existing switch creation and billing behavior continue; a sanitized error-class warning is best-effort only.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

SDG-008 - Indexed latency, log volume and capacity gate	MUST-PASS / P1
Preconditions / build / role	Production-equivalent schema/indexes; representative switch traffic; endpoint latency, DB and disk monitoring.
Test data	At least 1,000 switch creates plus seven-day replay; compare flag absent versus log mode.
Numbered steps	1. Confirm idx_user_type_connection_datetime range plan. 2. Measure query p50/p95/p99 and rows examined. 3. Compare endpoint latency/locks. 4. Measure warnings/day and log bytes. 5. Check DB CPU/connections and 5xx.
Expected result	The detector remains a bounded range lookup, adds no material endpoint/DB regression, and log growth is controlled. Any meaningful lock/latency/error increase blocks enforcement.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

SDG-009 - Three-node log-mode smoke and rollback	MUST-PASS / P0 / PROD
Preconditions / build / role	Explicit approval; identical controller/flag on app-1/app-2/app-3; controller rollback hash 2c1099563c4d3e781d77dc22aa3520c3322da1e9dd24aa8def5e236656c99368.
Test data	Node-local unauthenticated route smoke plus organic switch traffic; redacted aggregate evidence only.
Numbered steps	1. Verify controller, flag and backup hashes. 2. Verify PHP-FPM 8.2/8.4 active. 3. Confirm both create routes return expected 401 without auth. 4. Observe organic warnings/errors and API status. 5. Remove flag to stop detection; restore backup/reload only if code rollback is required.
Expected result	All nodes remain identical and healthy. Flag removal immediately disables the detector without changing calls or money. Controller restore is available and syntax-verified.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

RELEASE TEST APPENDIX SDC

Switch Duplicate Same-Channel Guard V2 - Log Mode

Backend observation-only refinement - 2026-07-23. V2 removes the broad detector from both call-creation transactions. It evaluates only an exact switchToAudio/Video call ID after the request supplies the active channel, validates the participants, and requires an older started, open, same-pair/type/channel switch leg. It never reuses, rejects, ends or settles a row and never changes wallets, ledgers, responses or notifications.

Release gate	SDC-001 through SDC-007 are MUST-PASS for v2 log mode. This remains evidence collection, not enforcement. The seven-day replay must retain all 14 strict duplicate pairs, production errors must remain zero, and no log result may be treated as a proven double bill without settled-row and ledger reconciliation.

SDC-001 - Creation routes return to pre-detector behavior

MUST-PASS / P0

Preconditions / build / role	V2 controller; flag both absent and present; call_female_user and call_male_user; current Android requests without channelName.
Test data	Normal calls, switches and retries at 0, 44, 45 and 46 seconds for audio/video and both initiator directions.
Numbered steps	1. Compare the creation blocks byte-for-byte with the pre-SDG controller. 2. Execute every fixture with flag Off and On. 3. Record query count, JSON, rows and existing CALL_DEDUP_HIT behavior.
Expected result	The broad SWITCH_DUP_CREATE query/log is absent. Creation responses, row count and the existing 45-second dedupe are unchanged; the v2 flag adds no creation-transaction query.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

SDC-002 - Exact same-channel started/open collision is observed only

MUST-PASS / P0

Preconditions / build / role	V2 flag present; valid individual switch call row and exact participants; older same-pair/type/channel switch row is started and open.
Test data	Valid switchToVideo <call_id> and switchToAudio <call_id> notifications, including a confirmed strict duplicate fixture.
Numbered steps	1. Send the switch request notification with its real current call ID and channel. 2. Capture the log and notification result. 3. Verify both rows and all financial fields before/after. 4. Settle only in isolated QA when required.
Expected result	Exactly one SWITCH_DUP_CHANNEL_GUARD_WOULD_REUSE log records version=2, same_channel=true, prior_started=true and prior_open=true. Notification and existing data behavior are unchanged.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

SDC-003 - Different-channel, orphan and sequential legs are excluded

MUST-PASS / P0

Preconditions / build / role	V2 flag present; controlled switch rows representing the false positives observed by v1.
Test data	Different channel; prior unstarted/open; prior started/ended; different payer, creator or type; random connection; older than two hours; legitimate video-audio-video sequence.
Numbered steps	1. Send each exact switch notification. 2. Inspect v2 logs and current call/notification outcomes. 3. Reconcile that no existing row changes. 4. Repeat both participant orientations.
Expected result	No v2 WOULD_REUSE log appears. Legitimate retries and sequential switching continue normally; no row is reused or blocked.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

SDC-004 - Current Android two-stage contract remains compatible	MUST-PASS / P0
Preconditions / build / role	Current supported Android build; creation request has no channelName; subsequent switch notification contains call ID and active channel.
Test data	Male/female audio and video activities, accepted/declined switches, notification failure and retry.
Numbered steps	1. Capture creation fields. 2. Confirm the returned call ID is embedded in switchToAudio/Video. 3. Confirm the existing channel backfill and push response. 4. Compare against the previous controller.
Expected result	No APK/API contract change is required. Non-switch, accepted, declined and malformed messages do not run the v2 DB lookup; switch notification behavior remains compatible.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

SDC-005 - Invalid or spoofed switch messages fail closed for detection	MUST-PASS / P0
Preconditions / build / role	QA endpoint access matching production middleware; v2 flag present; no real notification recipient required for negative fixtures.
Test data	Zero/non-numeric/missing call ID; wrong type; non-switch row; wrong participants; wrong connection type; missing channel; DB read/log failure.
Numbered steps	1. Submit every malformed/mismatched fixture. 2. Confirm no candidate log and no extra row mutation. 3. Inject DB and logger failures. 4. Inspect error fields for sensitive values.
Expected result	No false candidate is emitted. Detector exceptions are caught and log only sanitized error class; no user ID or channel value is added to v2 logs, and existing notification behavior proceeds.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

SDC-006 - Replay, index and capacity gate	MUST-PASS / P1
Preconditions / build / role	Production-equivalent 43M-row schema and representative traffic; redacted seven-day settled-switch replay.
Test data	32,415 positive settled switch rows; 14 strict duplicate pairs; at least 1,000 switch-message evaluations.
Numbered steps	1. Confirm PRIMARY lookup for current call. 2. EXPLAIN the candidate query. 3. Replay seven days. 4. Measure p50/p95/p99, query count, rows examined, DB CPU, endpoint 5xx and log bytes.
Expected result	Candidate query uses idx_user_type_connection_datetime range access (baseline estimate three rows). Replay detects 14/14 strict pairs; current baseline is 79 settled candidates. No material latency, lock, capacity or error regression is allowed.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

SDC-007 - Three-node activation and rollback	MUST-PASS / P0 / PROD
Preconditions / build / role	Explicit approval; fresh v1 backup; identical staged v2 controller and flag on app-1/app-2/app-3; PHP 8.2/8.4 available; any later compatible live patches identified.
Test data	Node-local 401 smoke for both creation routes and remaining-time; organic switch notifications; rollback drill without real calls.
Numbered steps	1. Verify old/new/backup hashes and ownership. 2. Syntax-check and atomically activate all nodes. 3. Gracefully reload both FPM versions. 4. Check status/error/log aggregates. 5. Disable the flag for immediate rollback. 6. If code rollback is required after another live patch, remove only v2 from the current live file.
Expected result	All nodes remain identical and healthy with zero detector errors after the reload settles. Flag removal disables v2 immediately. Never restore an older whole-file backup when that would remove a later compatible production fix; use a freshly reviewed surgical rollback.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

RELEASE TEST APPENDIX SDCV3

Switch Duplicate Backfill-Identity Guard V3 - Log Mode

Backend observation-only correction - 2026-07-23. V3 keeps the exact same-channel, started/open candidate rule but observes the row selected by channel backfill instead of blindly trusting the older call ID repeated in the switch notification. Invalid backfill identities fall back to the message call ID. It never reuses, rejects, ends, settles, debits or credits any row.

Release gate	SDCV3-001 through SDCV3-005 are MUST-PASS for log mode. Enforcement remains prohibited. A V3 hit is only candidate evidence until both call rows and their independent debit/income ledgers are reconciled.

SDCV3-001 - Known older-message-ID duplicate is observed — MUST-PASS / P0

Preconditions / build / role	V3 flag present; first video-switch row is started/open; a retry creates a newer same-pair/type row; the retry notification still embeds the first row ID.
Test data	Reproduce the verified ordering: newer row is selected by channel backfill and receives the same Agora channel while the notification carries the older ID.
Numbered steps	1. Create the first switch row and start it. 2. Create the newer switch row after the existing 45-second dedupe window. 3. Send the retry switch notification with the older ID. 4. Inspect the selected backfill row, guard log, response and financial state.
Expected result	Exactly one candidate log has version=3, observation_identity=channel_backfill_row and message_call_id_matches=false. Neither row, wallet, ledger, response nor notification behavior changes.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

SDCV3-002 - Valid normal switch keeps V2 precision — MUST-PASS / P0

Preconditions / build / role	V3 flag present; normal current Android switch notification; the backfilled row and message call ID identify the same valid switch leg.
Test data	Audio and video, both participant orientations, with and without an older same-channel started/open leg.
Numbered steps	1. Send each exact switch notification. 2. Verify the backfilled row ID and message ID. 3. Inspect candidate/error logs. 4. Compare database rows, JSON and FCM behavior with V2.
Expected result	The same V2 candidate decision is preserved. A true candidate logs version=3 with message_call_id_matches=true; a non-candidate stays silent. No functional response or data change occurs.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

SDCV3-003 - Invalid backfill identity safely falls back — MUST-PASS / P0

Preconditions / build / role	V3 flag present; controlled race where the selected backfill row is non-switch, wrong type/participants/connection, or has a different persisted channel.
Test data	Each invalid preferred-row condition plus a valid and invalid message-call-ID fallback.
Numbered steps	1. Arrange each invalid preferred identity. 2. Send the exact switch message. 3. Confirm persisted-channel validation. 4. Inspect fallback query, candidate/error logs and all stored rows.
Expected result	V3 never evaluates the invalid preferred row as a duplicate. It performs at most one primary-key fallback read; valid message identity preserves V2 behavior, invalid identity stays silent, and all failures remain fail-open.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

SDCV3-004 - Negative, compatibility and performance regression	MUST-PASS / P1
Preconditions / build / role	Current Android build and production-equivalent user_calls indexes; V3 flag On and Off.
Test data	Non-switch/terminal/malformed messages, missing channel, legacy creation requests without channelName, 1,000 valid switch notifications and the seven-day duplicate replay.
Numbered steps	1. Run all negative messages. 2. Compare creation routes byte-for-byte. 3. EXPLAIN both primary-key and unchanged candidate reads. 4. Measure query count, p50/p95/p99, DB CPU, logs and 5xx. 5. Replay known duplicates.
Expected result	Creation is unchanged. Normal V3 uses the same two reads as V2; only invalid preferred identity adds one primary-key fallback. The candidate query keeps its indexed bounded plan. No material latency, capacity or error regression is allowed.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

SDCV3-005 - Three-node health, disable and code rollback	MUST-PASS / P0 / PROD
Preconditions / build / role	Explicit approval; exact current-generation backup and identical V3 candidate on app-1/app-2/app-3; PHP 8.2/8.4; node-local observation flag.
Test data	Hashes, ownership/mode, syntax, FPM health, organic affected-route traffic and aggregate logs; no synthetic notification send.
Numbered steps	1. Verify old/new/backup hashes. 2. Activate atomically on all nodes. 3. Gracefully reload both FPM versions. 4. Confirm services and settled organic traffic. 5. Remove the flag to disable observation if errors appear. 6. Restore only the exact fresh backup for code rollback.
Expected result	All nodes are byte-identical and healthy, detector errors stay zero, and settled traffic has no new V3-correlated 5xx. Flag removal immediately disables guard reads/logs. Exact backup restoration returns the old hash without losing unrelated live changes.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

Call-Status Terminal Guard

Backend compare-and-set regression coverage - 2026-07-23. Production remains shadow-only. A concurrent live change moved the identical three-node controller to SHA-256 2b3f9347e9cbab4da4809941d01245596a338d4bf54370174fb42e875e652245, so any functional candidate must be rebuilt from that exact current generation and re-reviewed before activation.

Release gate	CSG-001 through CSG-009 are MUST-PASS before release. The current shadow observed 204 connected-call terminal candidates across all three nodes by 12:05 IST with zero shadow-log failures, but the observation window was only about nine hours. Shadow evidence does not authorize deployment. Production activation and PHP-FPM reloads require a separate exact approval after the full-window review.

CSG-001 - Normal receiver Decline closes an unanswered ring

MUST-PASS / P0

Preconditions / build / role	Fresh unanswered individual call; valid caller and receiver; stable network; staging/test backend and two physical Android devices.
Test data	One direct audio call and one direct video call.
Numbered steps	1. Start a call and confirm the receiver is ringing. 2. Tap Decline before connection. 3. Retry a new direct call to the same receiver immediately. 4. Inspect the first test row and API result.
Expected result	The first row records rejected plus ended_time atomically; the caller leaves Connecting; the receiver is immediately reachable; terminal_applied=true and guard_blocked=false.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

CSG-002 - Caller cancel and timeout close the ring once

MUST-PASS / P0

Preconditions / build / role	Fresh unanswered individual calls; caller and receiver test accounts; missed-call evidence available without sending to real users.
Test data	One caller-cancel case and one full ring-timeout case.
Numbered steps	1. Start a call. 2. Cancel before Answer. 3. Repeat with a ring timeout. 4. Inspect status responses, call rows and missed-notification dedupe evidence. 5. Retry each status once.
Expected result	The winning not_answered update closes each unanswered row once. Only the first winning update is eligible for a missed push; duplicate delivery changes no fields and sends no second push.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

CSG-003 - Late reject cannot relabel a connected call	MUST-PASS / P0
Preconditions / build / role	Connected test call with started_time set and no end_reason; authenticated participant; controlled delayed request.
Test data	Audio and video calls, each with one delayed rejected request.
Numbered steps	1. Connect the call. 2. Submit delayed rejected for the exact call ID. 3. Keep the call connected briefly. 4. End normally. 5. Reconcile timing, reason and financial outcome.
Expected result	The delayed request settles HTTP 200 with success=true, terminal_applied=false, guard_blocked=true and end_reason=null. Connected fields stay unchanged until normal End, which remains successful and billable.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

CSG-004 - Late terminal status cannot relabel a completed call	MUST-PASS / P0
Preconditions / build / role	Normally completed connected test calls with valid start/end timing and settlement evidence.
Test data	Fresh completed rows for delayed rejected, not_answered and failed.
Numbered steps	1. Submit each delayed terminal reason against its exact completed test row. 2. Capture status and JSON. 3. Compare all call timing/reason fields and missed-push evidence before and after.
Expected result	Every request settles without a client retry loop. No completed row is relabeled, no timing or financial field changes, and no duplicate missed notification is produced.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

CSG-005 - First terminal result remains idempotent	MUST-PASS / P0
Preconditions / build / role	Fresh unanswered test ring and an authenticated participant.
Test data	rejected followed by duplicate rejected and then not_answered.
Numbered steps	1. Submit rejected. 2. Resubmit rejected. 3. Submit not_answered for the same call. 4. Compare actor, reason, ended_time, response and notification evidence after every request.
Expected result	The first compare-and-set wins. Later requests return the existing reason and never overwrite actor, reason or timing; notification side effects are not duplicated.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

CSG-006 - Attend wins before a stale terminal retry

MUST-PASS / P0

Preconditions / build / role	Two physical devices ready to connect; delayed status delivery can be controlled.
Test data	Audio and video calls with a queued rejected/not_answered retry.
Numbered steps	1. Answer and join Agora. 2. Confirm Attend records started_time. 3. Deliver the delayed ring-terminal request. 4. Continue briefly and end normally. 5. Reconcile backend and client state.
Expected result	Attend records connection; the terminal compare-and-set affects zero rows; both clients remain connected; the call settles normally with no stale terminal label.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

CSG-007 - Terminal-before-Attend keeps current-client compatibility

MUST-PASS / P0

Preconditions / build / role	Controlled race environment and current production-compatible Android build.
Test data	Genuine peer join delivered immediately after a pre-connect terminal result.
Numbered steps	1. Stamp a pre-connect terminal result. 2. Deliver a genuine Agora peer join and Attend immediately afterward. 3. Observe both clients. 4. End and reconcile the final row.
Expected result	Existing Attend healing remains unchanged: the connected row has started_time and the pre-connect terminal fields are cleared. No client remains in Agora while the backend silently refuses Attend.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

CSG-008 - Offline WorkManager retry settles safely	MUST-PASS / P0
Preconditions / build / role	Android build containing the current reject WorkManager path; controllable networking.
Test data	Unanswered retry and a separate race where the call connects before networking returns.
Numbered steps	1. Remove receiver networking during Decline. 2. For the race variant, allow the call to connect before restoring network. 3. Restore network. 4. Let WorkManager deliver. 5. Observe retry count, response and row.
Expected result	For an unanswered ring, the retry closes it. For an already-connected call, the retry is a successful guarded no-op. WorkManager does not loop indefinitely or relabel the call.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

CSG-009 - Three-node compatibility, capacity and rollback gate	MUST-PASS / P0 / PROD
Preconditions / build / role	Separate explicit deployment/reload approval; candidate rebuilt from the then-current identical live controller; fresh per-node backups; PHP 8.2/8.4.
Test data	Exact hashes, node-local unauthenticated contract smoke, test-environment call cases, redacted organic metrics and a non-production rollback rehearsal.
Numbered steps	1. Verify live/candidate/backup hashes and syntax on all nodes. 2. Confirm the terminal update is PRIMARY-key bounded. 3. Activate only the isolated reviewed hunk. 4. Reload both FPM versions. 5. Check 401 smoke, FPM health, fatals, guard/shadow errors and endpoint latency. 6. Rehearse surgical rollback outside production.
Expected result	All nodes are identical and healthy; existing API compatibility is preserved; no unbounded query or material latency/error regression occurs; rollback restores the exact pre-change behavior without removing unrelated live patches.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

RELEASE TEST APPENDIX RBM

Incoming-Ring Swipe Backstop

Android regression coverage - 2026-07-23. The fix starts the existing bounded ring watcher for OneSignal fallback delivery and for FCM rings shown while Hima is foreground and unlocked. A no-action task swipe must report rejected so a free creator is not left temporarily busy. Normal connected-call ending is unchanged.

Release gate	RBM-001 through RBM-006 are MUST-PASS on physical devices before shipping. The backend call-status terminal guard (CSG) must be active first; without it, a delayed durable reject could relabel a call that connected during the retry window. Source compile success alone does not authorize APK release.

RBM-001 - FCM foreground-unlocked swipe releases creator

MUST-PASS / P0

Preconditions / build / role	Current Android candidate on two physical devices; receiver app open and unlocked; backend terminal guard active in staging; stable network.
Test data	Female receiver and male receiver; one audio and one video direct call each.
Numbered steps	1. Keep the receiver inside Hima with the phone unlocked. 2. Start the call and confirm the in-app accept screen appears with no duplicate system popup. 3. Open Recents and swipe Hima away without Answer or Decline. 4. Immediately call the same receiver again. 5. Inspect client logs, first call row and API result.
Expected result	The bounded backstop starts without a notification, onTaskRemoved reports rejected once, the caller exits Connecting, and the receiver is selectable/reachable again within five seconds. The first row is closed and no duplicate ring card appears.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

RBM-002 - OneSignal-only fallback swipe releases creator

MUST-PASS / P0

Preconditions / build / role	Physical Android device with FCM delivery suppressed in a controlled test environment; OneSignal fallback enabled; notification and full-screen permissions recorded.
Test data	Locked/background and process-cold cases for female and male receivers; audio and video.
Numbered steps	1. Suppress only the primary FCM delivery. 2. Send the test call through the normal OneSignal fallback. 3. Confirm one CallStyle ring and Telecom registration. 4. Swipe the task away without action. 5. Retry the receiver immediately and inspect the call row/logs.
Expected result	OneSignal starts the foreground watcher, exactly one incoming-call card is visible, the swipe reports rejected once, and the receiver is free within five seconds. If Telecom/FGS dispatch is denied, the display fallback remains visible and the failure is logged.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

RBM-003 - Accepted call is never rejected by task swipe

MUST-PASS / P0

Preconditions / build / role	Physical devices; accepted audio and video calls; backend row has started_time.
Test data	FCM and OneSignal-fallback entry variants, both receiver genders.
Numbered steps	1. Start and Answer each call. 2. Confirm both peers join Agora and started_time is set. 3. Swipe Hima from Recents while the watcher is still inside its 35-second lifetime. 4. Inspect logs and row. 5. End the call normally and reconcile the result.
Expected result	wasRingAcceptedFor blocks the swipe reject. No rejected/not_answered write is attempted, the connected row is not relabeled, and normal connected-call teardown and settlement remain unchanged.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

RBM-004 - Decline survives offline and app death for both receivers	MUST-PASS / P0
Preconditions / build / role	Backend terminal guard active; WorkManager inspection available; network can be toggled.
Test data	Female and male receiver Decline cases plus one delayed-retry race that connects first.
Numbered steps	1. Disable receiver networking while ringing. 2. Tap Decline and close the app. 3. Restore network and allow WorkManager to run. 4. Repeat with a controlled race where started_time wins before the retry. 5. Inspect unique work, API response and row.
Expected result	One unique reject job per call survives process death. An unanswered row closes when network returns; an already-connected row is a guarded no-op. There is no retry loop, duplicate notification, or connected-call relabel.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

RBM-005 - Lifecycle, notification and capacity regression	MUST-PASS / P1
Preconditions / build / role	Android 8, 13, 14 and 15 physical/emulated coverage; profiler/log collection enabled.
Test data	100 foreground-unlocked rings, 100 OneSignal fallback rings, and 100 controlled same-call FCM+OneSignal simultaneous/OneSignal-first deliveries without provider fan-out to production users.
Numbered steps	1. Repeat each ring path and cancel normally, swipe, Answer, and let it time out. 2. Count Telecom addNewIncomingCall dispatches, service instances, notifications, API calls and WorkManager jobs. 3. For dual delivery, verify both provider callbacks race on the same call ID and only one Telecom registration wins. 4. Measure main-thread stalls, memory and battery wake time. 5. Verify service teardown after 35 seconds.
Expected result	At most one bounded service instance exists per process, no network work is added to the main thread, each same-call dual delivery dispatches addNewIncomingCall exactly once, the foreground-open path adds no system popup, and all service callbacks are removed at teardown. No material latency, memory, battery or notification regression occurs.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

RBM-006 - Compatibility, rollback and production smoke gate	MUST-PASS / P0 / PROD
Preconditions / build / role	Signed QA build first; explicit release approval; backend guard already verified active; test accounts only; rollback build retained.
Test data	Fresh install and upgrade install, notification denied/allowed, keyguard on/off, process warm/cold, both app flavors.
Numbered steps	1. Complete RBM-001 through RBM-005 on the signed QA build. 2. Verify development and production Kotlin compiles. 3. Install the production candidate on controlled devices. 4. Run one FCM and one OneSignal smoke call per receiver gender. 5. Compare busy duration, crashes, ANRs and call-status errors. 6. Rehearse rollback to the prior APK.
Expected result	All compatibility states retain a visible actionable ring or safe display fallback, free receivers are not stranded busy after a swipe, connected calls remain protected, and no new crash/ANR/call-status error appears. Rollback restores prior app behavior without a server rollback.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

RELEASE TEST APPENDIX SSV4

Switch-session Identity and Settlement Shadow V4

Android adds an immutable root call ID, a stable UUID per pending audio/video switch intent, and the existing Agora channel to switch-row creation. The backend flag is absent by default. When explicitly enabled, it performs bounded reads and writes redacted decisions only. It never reuses, rejects, ends, settles, debits or credits a call.

Release gate	SSV4-001 through SSV4-006 are MUST-PASS before shadow deployment. Shadow evidence must include true positive and false-positive reconciliation before a separately designed, reviewed and approved atomic enforcement change. Log mode itself is not the financial cure.

SSV4-001 - Stable Android switch identity	MUST-PASS / P0
Preconditions / build / role	Production-flavor unit build; one active call with a positive initial call ID; male/female audio and video screens.
Test data	Same-target retry, opposite-target transition, accepted, declined, invalid target and activity-side replacement of the active switch-leg ID.
Numbered steps	1. Capture the initial root. 2. Begin video twice. 3. Begin audio. 4. Complete the matching intent. 5. Replace the activity call ID and begin another switch. 6. Repeat on all four screens.
Expected result	Same-target retry keeps one UUID; a new target or completed later intent gets a new UUID; root remains the original call ID. No timer, dialog, FCM, call-row or billing behavior changes.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

SSV4-002 - Legacy and flag-Off compatibility	MUST-PASS / P0
Preconditions / build / role	Backend candidate installed with switch_session_v4_shadow.flag absent; current and legacy APK request bodies.
Test data	Normal and switch callFemaleUser/callMaleUser requests with no V4 fields, partial fields, and complete fields.
Numbered steps	1. Remove/confirm absence of the flag. 2. Replay each request class. 3. Compare status, JSON, inserted row, channel, response time and logs with the pre-V4 build. 4. Confirm no V4 database query or log executes.
Expected result	All requests retain existing behavior. Optional fields do not break legacy clients. Flag Off returns before V4 queries/logging and never changes the selected or created row.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

SSV4-003 - Creation shadow sees the proven duplicate signature	MUST-PASS / P0
Preconditions / build / role	Shadow flag On in isolated QA; valid non-switch root, same participants/channel, prior started/open same-target switch leg; no financial fixtures in production.
Test data	Recreate the verified 82-second-later second video-row request and retry the exact request UUID once.
Numbered steps	1. Create and start switch leg A. 2. Send switch leg B creation with root, channel and UUID. 3. Retry B using the same UUID. 4. Inspect redacted V4 logs and unchanged API/database behavior.
Expected result	Creation shadow identifies the prior active same-target leg. Same-request retries share one request fingerprint. Calls are still created exactly as before in shadow mode; no row is reused or blocked.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

SSV4-004 - Untrusted root and malformed contract fail open	MUST-PASS / P0
Preconditions / build / role	Shadow flag On; controlled roots for wrong participants, switch root, missing channel and nonexistent ID.
Test data	Missing/zero root, non-UUID request key, wrong target, wrong participants, switch-leg root, absent root and blank channel.
Numbered steps	1. Send each malformed or forged request. 2. Inspect root validation decision and error count. 3. Verify response, call-row behavior, wallets and ledgers. 4. Repeat with a valid request afterward.
Expected result	Invalid contracts are observed only and never trusted for enforcement. No V4 log contains PII, a call ID or a channel. Every failure is fail-open and the later valid request remains healthy.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

SSV4-005 - Settlement recall and transition precision	MUST-PASS / P0
Preconditions / build / role	Isolated QA rows for a same-video overlap and a legitimate video-audio-video sequence; settlement APIs run normally.
Test data	Prior active same-mode leg, prior billed same-mode leg overlapping the newer row, non-overlapping re-entry, and immediately prior opposite-mode transition.
Numbered steps	1. Settle each current row through both settlement routes. 2. Reconcile call rows plus independent caller-debit and creator-income ledgers. 3. Inspect one redacted settlement decision per winning settlement. 4. Retry settlement.
Expected result	True same-mode overlaps are classified as candidates; the intervening opposite mode is classified as legitimate; non-overlapping re-entry is not blocked. Shadow never changes settlement, money, response or retry behavior.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

SSV4-006 - Performance, three-node health and rollback	MUST-PASS / P0 / PROD
Preconditions / build / role	Separate deployment approval; exact live-source integration and rollback backup per node; PHP 8.2/8.4; indexed production-equivalent user_calls data.
Test data	EXPLAIN root PK and bounded candidate reads; representative switch load; node hashes, flags, logs, FPM, route 2xx/4xx/5xx, CPU, memory, DB latency and disk growth.
Numbered steps	1. Prove indexed plans and baseline latency. 2. Deploy identical code with flag absent. 3. Reload both FPM versions after approval. 4. Enable shadow separately after approval. 5. Observe organic traffic. 6. Disable by flag removal and verify. 7. Restore only exact current-generation backups if needed.
Expected result	No material route/DB/capacity regression; all nodes remain identical and healthy; V4 errors stay zero. Flag removal stops reads/logs without code rollback. Enforcement remains Off until precision and recall are proven and separately approved.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

RELEASE TEST APPENDIX FBUSY

Creator Must Not Remain Falsely Busy

Android closes the exact unaccepted ring when its task is swiped away, releases stale Activity and Telecom ownership, and durably closes a row when a genuinely concurrent ring is rejected. Accepted and newer calls are protected by exact sender and call-ID guards.

Release gate	FBUSY-001 through FBUSY-005 are MUST-PASS on production-flavor builds before APK rollout. Test audio and video in both directions on at least one affected OEM device. Confirm normal calls and billing are unchanged.

FBUSY-001 - Original swipe-away then immediate redial — MUST-PASS / P0

Preconditions / build / role	Production-flavor APK containing the fix; creator availability enabled; caller and creator test accounts; affected OEM device when available.
Test data	Audio and video; FCM delivery and OneSignal fallback delivery; app foreground, background and killed.
Numbered steps	1. Caller starts a direct call. 2. While creator is ringing, swipe Hima away from recents. 3. Within 1-3 seconds call the same creator again. 4. Repeat each delivery and lifecycle combination. 5. Inspect redacted call state.
Expected result	The first ring, notification and Telecom entry end. Its row becomes terminal. The second call rings the same free creator; caller does not receive false userBusy/unavailable and is not redirected to random matching.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

FBUSY-002 - Accepted call is never torn down by task removal — MUST-PASS / P0

Preconditions / build / role	Production-flavor APK on two devices; valid accounts and sufficient caller balance.
Test data	Accept from full-screen UI and notification Answer; audio/video; task swipe immediately after accept and after Agora joins.
Numbered steps	1. Start and accept the call. 2. Swipe Hima from recents at each timing boundary. 3. Observe call status, Telecom and server row. 4. End normally. 5. Verify duration, debit and creator income once.
Expected result	The accepted call is not converted to rejected/not_answered by the ring backstop. No matching live Telecom connection is destroyed. Normal connected-call teardown and single billing remain unchanged.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

FBUSY-003 - Old callback cannot clear a newer ring — MUST-PASS / P0

Preconditions / build / role	QA build capable of closely spaced calls from two callers; creator availability enabled.
Test data	Caller A ring followed by caller B ring; delayed old service callback; different sender and different call ID.
Numbered steps	1. Start caller A ring. 2. End A and immediately deliver caller B ring. 3. Trigger or observe A's delayed task-removal/service teardown. 4. Accept or decline B. 5. Repeat 30 times.
Expected result	The stale A callback fails its exact identity claim and does not stop B's ringtone, notification, Activity or Telecom entry. B remains actionable and receives the correct terminal state.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

FBUSY-004 - Real concurrent busy rejection closes its own row	MUST-PASS / P0
Preconditions / build / role	Creator already in a real connected call or a valid current ring; a second caller starts another call; backend terminal compare-and-set guard enabled.
Test data	Online delivery, network loss during status post and recovery; duplicate worker enqueue; audio/video and both recipient genders.
Numbered steps	1. Keep call A legitimately active. 2. Start call B to the same user. 3. Confirm B receives userBusy. 4. Interrupt recipient network before the terminal post, then restore it. 5. Inspect redacted B state and retry behavior.
Expected result	Call A is untouched. Call B alone becomes not_answered after inline delivery or bounded WorkManager retry. Duplicate delivery is idempotent; B does not remain open and cannot create a later false-busy state.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________

FBUSY-005 - Late retry, performance and normal-call regression	MUST-PASS / P0
Preconditions / build / role	Production-flavor unit suite passing; QA backend with terminal compare-and-set; WorkManager and API telemetry available.
Test data	Terminal retry after a row has already started; 100 sequential swipe/redial cycles; normal accept/end, decline and timeout calls.
Numbered steps	1. Delay a not_answered retry until after the target row is marked started in QA. 2. Send the retry. 3. Run repeated swipe/redial cycles. 4. Run nearby normal flows. 5. Compare latency, worker count, API errors and billing with baseline.
Expected result	The backend guard ignores a late ring-terminal write on a started call. Only real busy rejections add one unique bounded worker. There is no main-thread network work, worker duplication, material call latency, call loss, revenue change or billing duplication.
Execution	Status: NOT RUN | Evidence: ____________________ | Defect: __________ | Retest: __________